
INTERVIEW PREPARATION GUIDE

100 React Native Interview Q&A

Theory · Strong Answers · Example Code · Architecture Diagrams

100
Questions

10
Sections

85+
Diagrams

6
Languages

Prepared for Mohammed Rinshad M I

React Native Engineer · iOS & Android · AI-Powered Mobile Apps · 2026



How to Use This Guide

This guide turns your resume into **100 realistic, interview-ready questions** across the full stack you actually work in — React Native, TypeScript, native modules, real-time, AI/LLM features, backend, payments and DevOps. Every question is answered in four layers so you understand it, not just memorise it:

THEORY

The concept and the *why* — what the interviewer is really probing for.

ANSWER

A natural, confident spoken answer you can deliver out loud. **Bold** terms are keywords to land.

CODE

A concise, correct example — what good looks like in real code.

ARCHITECTURE

An ASCII diagram of the flow or system, so you can draw it on a whiteboard.

PRO TIP

Read the **answers out loud**. Interviews reward fluent delivery, not perfect recall. When you give a number from your resume (35% latency cut, 30% faster delivery), have a one-line 'how I measured it' ready for the follow-up.

Contents

1	React Native Core & Fundamentals Components, JSX, hooks, lists, layout, platform code and re-renders.	Q1–Q10
2	TypeScript & JavaScript (ES6+) Types vs interfaces, generics, utility types, async, and the JS to TS migration.	Q11–Q20
3	State Management & Data Fetching Context, Redux Toolkit, thunks, selectors, React Query caching & mutations.	Q21–Q30
4	Animations, Gestures & Performance Threads, Reanimated worklets, gestures, 60fps, memoization & profiling.	Q31–Q40
5	Native Modules & the New Architecture Bridge vs JSI, TurboModules, Fabric, Codegen and Swift native modules.	Q41–Q50
6	Offline-First, SQLite, Geolocation & Background Tasks SQLite, outbox/sync, background geolocation, backoff and idempotency.	Q51–Q60
7	Real-Time: WebSockets, WebRTC & CRDTs WebSockets, reconnection, Yjs CRDTs, presence, WebRTC and scaling.	Q61–Q70
8	AI / LLM Features on Mobile Streaming chat, on-device Whisper, tool calling, RAG, embeddings & VAD.	Q71–Q80
9	Backend & Data REST, Express, JWT rotation, SQL vs NoSQL, indexing, Redis & idempotency.	Q81–Q90
10	Payments, Releases, Testing & DevOps Payments & webhooks, signing, staged rollout, Fastlane, CI/CD, tests, Sentry.	Q91–Q100

1

React Native Core & Fundamentals

Components, JSX, hooks, lists, layout, platform code and re-renders.

Q1-Q10

Q1

REACT NATIVE CORE & FUNDAMENTALS

What is React Native and how does it actually render native UI from JavaScript?

THEORY

React Native lets you build mobile apps using **React** and JavaScript while rendering **real native UI components** (not a WebView). Your JS code runs on a separate **JS thread**, and it communicates with the native side (the **UI/main thread**) to mount and update actual platform widgets like `UIView` on iOS and `android.view.View` on Android. Historically this happened over an asynchronous serialized **bridge**, while the newer architecture (Fabric, TurboModules, JSI) replaces it with a synchronous JSI layer for direct calls.

STRONG ANSWER

I'd explain that React Native is a framework for building native mobile apps with React, where I write components in JavaScript and they map to real native views, not HTML in a browser. My JavaScript logic runs on the **JS thread**, and instructions to render or update UI are sent to the **native UI thread**. In the old architecture this used an asynchronous **bridge** that serialized messages, which could be a bottleneck for heavy interactions. The new architecture with **Fabric** and **JSI** removes that serialization, letting JS hold direct references to native objects for faster, more synchronous communication. So in practice you get near-native UI and performance while sharing one codebase across platforms.

CODE · TypeScript / TSX

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

// These JSX elements are NOT DOM nodes – they map to real native views.
// <View> -> UIView (iOS) / android.view.View (Android)
// <Text> -> UILabel (iOS) / TextView (Android)
export default function App(): JSX.Element {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Hello, native world</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  title: { fontSize: 20, fontWeight: 'bold' },
});
```

ARCHITECTURE / FLOW

JS THREAD		NATIVE / UI THREAD	
(your React code)		(UIView / android View)	
components, state	----->	mounts & updates real	
diffing, layout req	bridge	native UI widgets	
	<-----	touches, events	
/JSI			

Q2

REACT NATIVE CORE & FUNDAMENTALS

What are functional components and how does JSX work in React Native?

THEORY

A **functional component** is a plain JavaScript/TypeScript function that takes **props** and returns UI described in **JSX**. JSX is syntactic sugar: each tag compiles down to a `React.createElement` call that produces a lightweight element object describing what to render. Functional components are the modern default and, combined with **Hooks**, can hold state and side effects without needing classes.

STRONG ANSWER

I'd say a functional component is just a function that receives props and returns JSX describing the UI. JSX isn't HTML — it's compiled into `React.createElement` calls that create element objects React uses to build the tree. In React Native the JSX tags are native components like **View** and **Text** rather than `div` or `span`. I prefer functional components because they're concise, easier to test, and with Hooks they fully replace what we used to do with class lifecycles. One thing I always remember is that any string of text must be wrapped in a **Text** component, unlike on the web.

CODE · TypeScript / TSX

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

type Props = { name: string; online?: boolean };

// A functional component: function in, JSX out.
export function UserBadge({ name, online = false }: Props) {
  return (
    <View style={styles.row}>
      /* In RN, every piece of text must live inside <Text> */
      <Text style={styles.name}>{name}</Text>
      <Text>{online ? '■ online' : '■ offline'}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  name: { fontWeight: '600' },
});
```

Q3

REACT NATIVE CORE & FUNDAMENTALS

What is the difference between props and state in React Native?

THEORY

Props are read-only inputs passed from a parent to a child component; they flow **downward** and the child must not mutate them. **State** is data owned and managed *inside* a component that can change over time, and updating it (via a setter) triggers a **re-render**. The rule of thumb: props configure a component from the outside, state tracks things that change from within.

STRONG ANSWER

I'd describe props as the arguments a parent passes into a child — they're immutable from the child's perspective and create a one-way data flow downward. State, on the other hand, is internal data the component manages itself, and changing it through its setter causes a re-render. A common pattern is to keep state in a parent and pass both the value and an updater function down as props, which is essentially 'lifting state up'. The key interview point is that you should never directly mutate either; props are read-only, and state must always be updated through `setState` or the `useState` setter so React knows to re-render.

CODE · TypeScript / TSX

```
import React, { useState } from 'react';
import { View, Text, Button } from 'react-native';

// Child receives data + callback as PROPS (read-only here)
function Counter({ count, onAdd }: { count: number; onAdd: () => void }) {
  return (
    <View>
      <Text>Count: {count}</Text>
      <Button title="Add" onPress={onAdd} />
    </View>
  );
}

// Parent OWNS the state and passes it down
export function Screen() {
  const [count, setCount] = useState(0); // state lives here
  return <Counter count={count} onAdd={() => setCount((c) => c + 1)} />;
}
```

ARCHITECTURE / FLOW

```
PARENT (owns state)
+-----+
| count = 3 |
| setCount(...) |
+-----+
      |
      | (read-only, flows DOWN)
      v
+-----+
| CHILD |
| uses count |
| calls onAdd() --> triggers parent setState
+-----+
```

Q4**REACT NATIVE CORE & FUNDAMENTALS****How do useState and useEffect work, and when does useEffect run?****THEORY**

useState declares a piece of reactive state and returns the current value plus a setter; calling the setter schedules a re-render. **useEffect** runs side effects (data fetching, subscriptions, timers) *after* render, and its **dependency array** controls when it re-runs: empty array means once on mount, listed deps mean run when any changes, and no array means after every render. Returning a function from the effect provides a **cleanup** that runs before the next effect and on unmount.

STRONG ANSWER

I'd explain that `useState` gives me a value and a setter, and updating it re-renders the component with the new value. `useEffect` is where I put side effects that shouldn't happen during render, like fetching data or subscribing to events. The dependency array is critical: an empty array runs it once on mount, including specific values runs it whenever those change, and omitting it runs after every render which is usually a mistake. I always return a cleanup function when I set up subscriptions or timers to avoid memory leaks, and I use the functional updater form of the setter when the new state depends on the previous one. A common gotcha I watch for is missing dependencies causing stale values.

CODE · TypeScript / TSX

```
import React, { useState, useEffect } from 'react';
import { Text } from 'react-native';

export function Clock() {
  const [now, setNow] = useState(() => new Date());

  useEffect(() => {
    // runs after mount because deps array is empty
    const id = setInterval(() => setNow(new Date()), 1000);
    return () => clearInterval(id); // cleanup on unmount
  }, []);

  return <Text>{now.toLocaleTimeString()}</Text>;
}
```

ARCHITECTURE / FLOW

```
render() ----> commit to screen ----> useEffect runs  
      ^                                         |  
      |                                         | (state/prop dep changes)  
      |                                         v  
      | cleanup() <--- before next effect / on unmount  
+-----+  
  
deps []       -> run once on mount  
deps [a, b]   -> run when a or b changes  
deps omitted -> run after EVERY render
```

Q5

REACT NATIVE CORE & FUNDAMENTALS

What are useRef, useMemo, and useCallback, and when would you use each?

THEORY

useRef holds a mutable value that persists across renders without triggering re-renders, commonly used to reference native components (to call methods like `focus` or `scrollTo`) or to store mutable instance-like data. **useMemo** caches the result of an expensive computation and recomputes it only when its dependencies change. **useCallback** memoizes a function identity so it stays stable between renders, which matters when passing callbacks to memoized children or to dependency arrays.

STRONG ANSWER

I'd say `useRef` is for values that need to survive re-renders but shouldn't cause one — like holding a reference to a `TextInput` so I can call `focus` on it, or keeping a mutable flag. `useMemo` memoizes a computed value so an expensive calculation only re-runs when its inputs change. `useCallback` is essentially `useMemo` for functions; it keeps a function's identity stable so I don't break the memoization of a child wrapped in `React.memo` or trigger effects unnecessarily. The important caveat I always mention is not to over-optimize — these have their own cost, so I reach for them when there's a real performance problem or a referential-equality requirement, not by default.

CODE · TypeScript / TSX

```
import React, { useRef, useMemo, useCallback } from 'react';
import { TextInput, Button, View } from 'react-native';

export function SearchBox({ items }: { items: string[] }) {
  const inputRef = useRef<TextInput>(null); // ref to native node

  // expensive work memoized – recomputes only when items change
  const sorted = useMemo(() => [...items].sort(), [items]);

  // stable function identity across renders
  const focusInput = useCallback(() => inputRef.current?.focus(), []);

  return (
    <View>
      <TextInput ref={inputRef} placeholder={`${sorted.length} items`} />
      <Button title="Focus" onPress={focusInput} />
    </View>
  );
}
```

ARCHITECTURE / FLOW

```
useRef    -> persists value, NO re-render   (e.g. .current = node)
useMemo    -> caches a VALUE, recompute on dep change
useCallback -> caches a FUNCTION identity, stable across renders

      deps change?
no  -> return cached result
yes -> recompute & cache new result
```

Q6

REACT NATIVE CORE & FUNDAMENTALS

What is the difference between FlatList and ScrollView, and what is list virtualization?

THEORY

ScrollView renders *all* of its children at once, so it's fine for small, finite content but expensive and memory-heavy for long lists. **FlatList** is built for large datasets because it **virtualizes**: it only renders the items currently visible (plus a small buffer) and recycles them as you scroll, keeping memory and render cost low. FlatList also provides built-in features like pull-to-refresh, pagination via onEndReached, and item separators.

STRONG ANSWER

I'd explain that ScrollView mounts every child immediately, which is great for a short form or a few items but will hurt performance and memory if you have hundreds of rows. FlatList solves that with virtualization — it only renders what's on screen plus a buffer and unmounts off-screen items as you scroll. For performance I always provide a stable **keyExtractor**, and for fixed-height rows I add **getItemLayout** so it can skip measurement. I also memoize the renderItem component to avoid unnecessary re-renders. The rule I follow is: known small list use ScrollView, large or dynamic data use FlatList.

CODE · TypeScript / TSX

```
import React from 'react';
import { FlatList, Text, View } from 'react-native';

type Item = { id: string; title: string };

export function MessageList({ data }: { data: Item[] }) {
  return (
    <FlatList
      data={data}
    />
  );
}
```


CODE · TypeScript / TSX (continued)

```

    keyExtractor={({item}) => item.id}          // stable keys
    renderItem={({ item }) => (
      <View style={{ padding: 12 }}>
        <Text>{item.title}</Text>
      </View>
    )}
    initialNumToRender={10}
    onEndReachedThreshold={0.5}
    onEndReached={() => {/* load more */}}
  />
);
}

```

ARCHITECTURE / FLOW

ScrollView: renders EVERYTHING (heavy for long lists)
 [1][2][3][4][5][6][7][8][9][10]...[1000] all in memory

FlatList (virtualized): only viewport + buffer rendered

```

+----- viewport -----+
... | [4] [5] [6] [7] | ... off-screen = recycled
+-----+
^ recycled                recycled ^

```

Q7

REACT NATIVE CORE & FUNDAMENTALS

How does styling work with StyleSheet, and how do you build layouts with Flexbox in React Native?

THEORY

React Native styles are written as JavaScript objects, typically created with **StyleSheet.create**, using camelCased properties and unitless numbers that represent density-independent pixels. Layout uses **Flexbox**, but with key differences from the web: the default **flexDirection** is **column** (not row), and every View is a flex container. You position children with **justifyContent** (main axis), **alignItems** (cross axis), and size them with **flex**.

STRONG ANSWER

I'd say there's no CSS file — styles are plain JS objects, and I use **StyleSheet.create** for them, which gives a tiny optimization and better validation. Properties are camelCase and numbers are unitless density-independent pixels. Layout is Flexbox, but the big gotcha versus the web is that the default **flexDirection** is **column**, so things stack vertically by default. I control alignment with **justifyContent** along the main axis and **alignItems** along the cross axis, and I use **flex** to distribute space proportionally. I also keep styles outside the render body so they aren't recreated on every render.

CODE · TypeScript / TSX

```

import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

export function Card() {
  return (
    <View style={styles.card}>
      <Text style={styles.label}>Total</Text>
      <Text style={styles.value}>$42.00</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  card: {

```

CODE · TypeScript / TSX (continued)

```

flexDirection: 'row',           // default would be 'column'
justifyContent: 'space-between', // main axis
alignItems: 'center',          // cross axis
padding: 16,
borderRadius: 8,
backgroundColor: '#f2f2f2',
},
label: { color: '#666' },
value: { fontWeight: '700' },
});

```

ARCHITECTURE / FLOW

```

flexDirection: 'row' (main axis -> horizontal)
+-----+
| [Total]      <space-between>    [$42] |
+-----+
justifyContent = along MAIN axis (-->)
alignItems     = along CROSS axis (|)

(default flexDirection in RN = 'column', stacks vertically)

```

Q8

REACT NATIVE CORE & FUNDAMENTALS

How do you write platform-specific code in React Native?

THEORY

React Native exposes the **Platform** module so you can branch at runtime: **Platform.OS** tells you ios or android, and **Platform.select** returns a value based on the platform. For larger divergences you can split files using the **.ios.js** / **.android.js** (or **.tsx**) extensions, and the bundler automatically picks the right one for each build. This keeps a shared codebase while letting you honor platform conventions.

STRONG ANSWER

I'd say there are a few levels. For small tweaks like padding or a color I use **Platform.OS** in a ternary or **Platform.select**, which returns the right value per platform. For a whole component or module that differs significantly between platforms, I create separate files with **.ios.tsx** and **.android.tsx** extensions and import without the extension — Metro resolves the correct one automatically. I might also check **Platform.Version** when behavior depends on the OS version. The goal is to share as much logic as possible and only diverge where the platforms genuinely need to behave differently.

CODE · TypeScript / TSX

```

import { Platform, StyleSheet } from 'react-native';

const styles = StyleSheet.create({
  header: {
    // runtime branch on the platform
    paddingTop: Platform.OS === 'ios' ? 44 : 24,
    ...Platform.select({
      ios: { shadowOpacity: 0.2, shadowRadius: 4 },
      android: { elevation: 4 },
      default: {},
    }),
  },
});

// File-based split (Metro auto-resolves the right one):
// Button.ios.tsx -> used on iOS
// Button.android.tsx -> used on Android

```

CODE · TypeScript / TSX (continued)

```
// import Button from './Button'; // no extension needed
export { styles };
```

ARCHITECTURE / FLOW

```
import Button from './Button'
  |
  | Metro bundler resolves by build target
  | /
  | iOS build      Android build
  | Button.ios.tsx Button.android.tsx
  |
Runtime branch: Platform.OS === 'ios' ? A : B
                 Platform.select({ ios, android, default })
```

Q9

REACT NATIVE CORE & FUNDAMENTALS

How do you handle touch interactions with Pressable, and what are the basics of navigation?

THEORY

For touch, modern React Native favors **Pressable**, a flexible primitive that exposes press states (pressed, hovered) and supports `onPress`, `onLongPress`, and styling based on interaction, superseding older components like `TouchableOpacity`. For moving between screens, the de-facto library is **React Navigation**, where you wrap the app in a **NavigationContainer**, define a navigator (such as a stack), and move around using the **navigation** prop's methods like `navigate`, `push`, and `goBack`.

STRONG ANSWER

I'd explain that for handling taps I reach for **Pressable** because it gives me fine-grained control — I can style based on the pressed state and handle `onPress`, `onLongPress`, and delayed presses, whereas the older `Touchable` components are more limited. For navigation, the standard is **React Navigation**: I wrap everything in a `NavigationContainer`, set up a stack navigator with my screens, and then each screen gets a **navigation** prop I use to call `navigate` or `goBack`, optionally passing params. I also use the **route** prop to read params on the destination screen. For type safety I define a param list type so `navigate` calls and route params are checked.

CODE · TypeScript / TSX

```
import React from 'react';
import { Pressable, Text } from 'react-native';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

type Stack = { Home: undefined; Details: { id: string } };
const Nav = createNativeStackNavigator<Stack>();

function Home({ navigation }: any) {
  return (
    <Pressable
      onPress={() => navigation.navigate('Details', { id: '42' })}
      style={({ pressed }) => ({ opacity: pressed ? 0.6 : 1, padding: 16 })}
    >
      <Text>Go to details</Text>
    </Pressable>
  );
}

export function App() {
  return (
```

CODE · TypeScript / TSX (continued)

```

<NavigationContainer>
  <Nav.Navigator>
    <Nav.Screen name="Home" component={Home} />
  </Nav.Navigator>
</NavigationContainer>
);
}

```

ARCHITECTURE / FLOW

```

NavigationContainer
|
+-- Stack.Navigator
|
+-- Screen 'Home' --navigation.navigate('Details',{id})-->
+-- Screen 'Details' (reads route.params.id)
    <-- navigation.goBack()

```

Q10

REACT NATIVE CORE & FUNDAMENTALS

Why are keys important in lists, what causes re-renders, and what is the component lifecycle in function components?

THEORY

Keys give React a stable identity for each item in a list so it can match elements between renders and update efficiently instead of re-creating them; using an array **index** as a key is risky when the list can reorder or change. A component **re-renders** when its state changes, its props change, or its parent re-renders, and React then reconciles the virtual tree to update only what changed. In function components the **lifecycle** (mount, update, unmount) is expressed through Hooks like `useEffect` rather than class methods such as `componentDidMount`.

STRONG ANSWER

I'd say keys are how React tracks list items across renders — with stable keys it can reorder and patch efficiently, but using the array index as a key causes bugs and wasted work when items are inserted, removed, or reordered, so I use a stable id. A component re-renders when its own state updates, when it receives new props, or when its parent re-renders, and React reconciles to apply only the actual differences. For lifecycle in function components, I map the class methods onto Hooks: mount and unmount are an effect with an empty dependency array plus its cleanup, and updates are effects keyed on specific dependencies. To cut unneeded re-renders I lean on **React.memo**, **useMemo**, and **useCallback** where it actually matters.

CODE · TypeScript / TSX

```

import React, { useEffect } from 'react';
import { FlatList, Text } from 'react-native';

type Todo = { id: string; text: string };

function Row({ item }: { item: Todo }) {
  useEffect(() => {
    // 'mount' for this row
    return () => { /* 'unmount' cleanup */ };
  }, []);
  return <Text>{item.text}</Text>;
}

export function TodoList({ todos }: { todos: Todo[] }) {
  return (
    <FlatList
      data={todos}

```

CODE · TypeScript / TSX (continued)

```
// STABLE id, not the array index, so reorders patch correctly
keyExtractor={item => item.id}
renderItem={({ item }) => <Row item={item} />}
  />
);
}
```

ARCHITECTURE / FLOW

Re-render triggers: state change | new props | parent re-render
|
v
reconcile virtual tree --> patch only what differs

Function component lifecycle via Hooks:

```
mount    -> useEffect(() => {...}, [])
update   -> useEffect(() => {...}, [dep])
unmount  -> return () => {...} (cleanup)
```

Keys: stable id -> matched & reused | index -> bugs on reorder

2

TypeScript & JavaScript (ES6+)

Types vs interfaces, generics, utility types, async, and the JS to TS migration.

Q11–Q20

Q11

TYPESCRIPT & JAVASCRIPT (ES6+)

Why do we use TypeScript in a React Native project and what problems does it solve over plain JavaScript?

THEORY

TypeScript is a **typed superset of JavaScript** that adds static types checked at compile time, then strips them out to produce plain JS. The core problem it solves is catching **whole categories of bugs before runtime** such as accessing undefined properties, passing wrong argument types, or misspelling fields. It also powers editor features like autocomplete, safe refactoring, and inline documentation.

STRONG ANSWER

In my projects TypeScript pays for itself mainly through safety and developer speed. Most of the bugs we used to hit in production were things like reading a property off an undefined object or passing the wrong shape into a component, and TypeScript catches those at compile time instead of letting them crash on a user's device. It also makes the codebase self-documenting, so when I open a function I immediately know what it expects and returns, and refactors like renaming a prop become safe because the compiler flags every place that breaks. The trade-off is some upfront typing effort, but on a 2.5-year codebase the payoff in fewer runtime crashes is clearly worth it.

CODE · TypeScript

```
// JavaScript: bug only shows up at runtime
function getName(user) {
  return user.profile.name; // crashes if profile is undefined
}

// TypeScript: error caught while typing
interface User {
  profile?: { name: string };
}

function getNameTs(user: User): string {
  // TS error: 'profile' is possibly 'undefined'
  // return user.profile.name;
  return user.profile?.name ?? "Guest"; // safe
}

const u: User = {};
console.log(getNameTs(u)); // "Guest", no crash
```

ARCHITECTURE / FLOW

```
JavaScript: write -> run -> CRASH on device (runtime bug)

TypeScript: write -> tsc type-check -> FAIL FAST in editor/CI
               |
               +--> emit plain JS -> run (fewer bugs)
```

Q12

TYPESCRIPT & JAVASCRIPT (ES6+)

What is the difference between type and interface in TypeScript, and when do you use each?

THEORY

Both **type aliases** and **interface** can describe the shape of an object, and for most object shapes they are interchangeable. The key differences: interface supports **declaration merging** (reopening to add fields) and is idiomatic for object and class contracts, while **type** can also alias unions, tuples, primitives, and mapped types. **type** uses intersection with the **and** operator to combine, whereas interface uses **extends**.

STRONG ANSWER

My rule of thumb is: use interface for object shapes and public contracts like component props or API models, and use type when I need something interface can't express, like a union, a tuple, or a mapped type. interface gives nicer error messages and supports declaration merging, which is useful when extending third-party types. type is more flexible for composing and for aliasing primitives or function signatures. In practice the teams I've worked on pick one convention for props and stay consistent, and reach for type whenever a union is involved.

CODE · TypeScript

```
// interface: object shape, extendable
interface Button {
  label: string;
}
interface Button {
  onPress: () => void; // declaration merging
}

interface IconButton extends Button {
  icon: string;
}

// type: unions, tuples, primitives - interface can't do these
type Status = "idle" | "loading" | "error";
type Point = [number, number];
type ID = string | number;

// composing with type
type Card = Button & { elevated: boolean };

const b: IconButton = { label: "Go", onPress: () => {}, icon: "x" };
```

ARCHITECTURE / FLOW

interface		type alias	
object shapes	ok	object shapes	ok
extends	ok	& intersection	ok
declaration merge	ok	declaration merge	NO
unions/tuples	NO	unions/tuples	ok
primitive alias	NO	primitive alias	ok

Q13

TYPESCRIPT & JAVASCRIPT (ES6+)

What are generics in TypeScript and why are they useful?

THEORY

Generics let you write **reusable, type-safe code that works over many types** by using a type parameter (commonly `T`) that the caller fills in. Instead of locking a function or component to one concrete type or falling back to any, the type flows through and is preserved on the output. This keeps full type checking and autocomplete while staying flexible.

STRONG ANSWER

Generics are how I avoid choosing between reusability and type safety. A classic example is a fetch wrapper or a list component: I don't want to write a separate version for users, products, and orders, but I also don't want to return any. With a type parameter the caller says what type they expect, and TypeScript carries it all the way through so the result is fully typed. I also use constraints with extends to guarantee the type has certain fields, like requiring an id on every item in a generic list.

CODE · TypeScript

```
// generic function: T is inferred from the argument
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}
const n = first([1, 2, 3]); // n: number | undefined
const s = first(["a", "b"]); // s: string | undefined

// generic with a constraint
interface HasId { id: string; }

function indexById<T extends HasId>(items: T[]): Record<string, T> {
  return items.reduce((acc, item) => {
    acc[item.id] = item;
    return acc;
  }, {} as Record<string, T>);
}

const map = indexById([{ id: "a", name: "Sam" }]);
// map["a"].name is fully typed
```

Q14

TYPESCRIPT & JAVASCRIPT (ES6+)

Which built-in utility types do you use most (Partial, Pick, Omit, Record, Readonly) and what do they do?

THEORY

Utility types **transform existing types instead of redefining them**, keeping a single source of truth. **Partial of T** makes all fields optional, **Pick of T and keys** selects a subset, **Omit of T and keys** removes a subset, **Record of K and V** builds an object map type, and **Readonly of T** makes fields immutable. Using them avoids duplicated, drifting type definitions.

STRONG ANSWER

These save me from copy-pasting types that then drift apart. Partial is great for update functions where the caller only sends a few fields, Pick and Omit let me derive props or DTOs from a master model, so when the model changes the derived types update automatically. Record is my go-to for dictionaries like a status-to-color map keyed by a union. Readonly I use to lock down config objects and state I don't want mutated. The big win is one source of truth: I define the entity once and shape everything else from it.

CODE · TypeScript

```
interface User {
  id: string;
  name: string;
  email: string;
  age: number;
}

// only some fields needed for an update
type UserUpdate = Partial<User>;

// derive a public profile from the model
type PublicUser = Pick<User, "id" | "name">;

// everything except the id
type NewUser = Omit<User, "id">;

// keyed map from a union to a value
type Status = "active" | "banned";
type StatusLabels = Record<Status, string>;

// immutable config
const config: Readonly<{ apiUrl: string }> = { apiUrl: "/v1" };
// config.apiUrl = "x"; // TS error: read-only
```

ARCHITECTURE / FLOW

```
User { id, name, email, age }
|
|-- Partial<User> -> { id?, name?, email?, age? }
+-- Pick<"id","name"> -> { id, name }
+-- Omit<"id"> -> { name, email, age }
+-- Readonly<User> -> all fields read-only

Record<"active"|"banned", string> -> { active: string;
                                     banned: string }
```

Q15**TYPESCRIPT & JAVASCRIPT (ES6+)****What are union and discriminated union types, and how does TypeScript narrow them?****THEORY**

A **union type** means a value can be one of several types, written with the pipe symbol. A **discriminated (tagged) union** adds a shared literal field, like a kind or status property, so TypeScript can tell the variants apart. **Narrowing** is when checks like typeof, in, or comparing the discriminant field let the compiler refine the type inside a branch, unlocking the right fields.

STRONG ANSWER

Discriminated unions are one of my favorite TypeScript features for modeling state, especially async UI. I give each variant a common tag like status, and then a switch on that tag narrows the type so in the success branch I get data and in the error branch I get the error message, with no risk of reading a field that doesn't exist there. The compiler enforces that I handle every case, and if I add a new variant later it flags the missing branch. It basically makes impossible states unrepresentable, which removes a whole class of runtime bugs.

CODE · TypeScript

```
type RequestState =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: string[] }
  | { status: "error"; message: string };

function render(state: RequestState): string {
  switch (state.status) {
    case "idle":
      return "Tap to load";
    case "loading":
      return "Loading...";
    case "success":
      return state.data.join(", "); // data is known here
    case "error":
      return state.message;         // message is known here
  }
}
```

ARCHITECTURE / FLOW

```
RequestState (discriminant = status)
|
switch (state.status)
|         |         |         |
idle    loading    success    error
()      ()      {data:string[]} {message:string}
              ^narrowed: only valid fields visible
```

Q16**TYPESCRIPT & JAVASCRIPT (ES6+)**

What is the difference between Promises and async/await, and how do you handle errors with each?

THEORY

Promises and async/await are **two syntaxes over the same underlying mechanism** for asynchronous work. Promises use `.then` and `.catch` chaining, while **async/await** lets you write asynchronous code that reads top-to-bottom like synchronous code. With Promises you handle errors via `.catch`; with async/await you use **try/catch**, which also catches errors thrown from awaited calls.

STRONG ANSWER

Under the hood they're the same, async/await is just sugar over Promises, but I default to async/await because it's far more readable and avoids deeply nested then chains. For error handling I wrap awaited calls in try/catch, and I make sure the catch distinguishes real failures from expected ones so I don't swallow errors silently. When I genuinely need things to run in parallel I still reach for Promise.all rather than awaiting sequentially, since awaiting one after another would be slower. The main gotcha I watch for is forgetting an await, which makes a function return a pending Promise instead of the value.

CODE · TypeScript

```
// Promise style
function loadUser(id: string): Promise<User> {
  return fetch(`/users/${id}`)
    .then((res) => res.json())
    .catch((err) => {
      console.error(err);
      throw err;
    });
}

// async/await style with try/catch
async function loadUserAsync(id: string): Promise<User | null> {
  try {
    const res = await fetch(`/users/${id}`);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return (await res.json()) as User;
  } catch (err) {
    console.error("Failed to load user", err);
    return null;
  }
}

// parallel, not sequential
async function loadTwo() {
  return Promise.all([loadUserAsync("1"), loadUserAsync("2")]);
}

interface User { id: string; name: string; }
```

Q17

TYPESCRIPT & JAVASCRIPT (ES6+)

Can you explain closures and how var, let, and const differ in scoping?

THEORY

A **closure** is a function that remembers the variables from the scope where it was created, even after that outer scope has returned. **var** is function-scoped and hoisted, while **let** and **const** are block-scoped and live in the temporal dead zone until declared. **const** prevents reassignment of the binding, though the referenced object can still be mutated.

STRONG ANSWER

Closures are everywhere in React Native, every hook callback and event handler closes over the variables in scope, which is exactly why stale-closure bugs happen with `useState` if your dependency array is wrong. On scoping, I always use `const` by default, `let` only when I need to reassign, and I avoid `var` entirely because its function scope and hoisting cause surprises. The classic example is a loop creating callbacks: with `var` they all share one variable and log the final value, but with `let` each iteration gets its own binding so they log correctly. Understanding that helps me reason about why a closure captured an old value.

CODE · TypeScript

```
// closure: counter remembers `count`
function makeCounter() {
  let count = 0;
  return () => ++count; // closes over count
}
const next = makeCounter();
next(); next(); // 1, 2

// var vs let in a loop
const varFns: (() => number)[] = [];
for (var i = 0; i < 3; i++) varFns.push(() => i);
console.log(varFns.map((f) => f())); // [3, 3, 3]
```

CODE · TypeScript (continued)

```
const letFns: (() => number)[] = [];
for (let j = 0; j < 3; j++) letFns.push(() => j);
console.log(letFns.map((f) => f())); // [0, 1, 2]

const obj = { n: 1 };
obj.n = 2; // ok, mutation allowed
// obj = {}; // error, reassignment blocked
```

ARCHITECTURE / FLOW

```
var -> function-scoped, hoisted -> loop shares 1 binding
      [i][i][i] all point to final i (=3)

let -> block-scoped per iteration
      [j=0][j=1][j=2] each callback keeps its own
```

Q18

TYPESCRIPT & JAVASCRIPT (ES6+)

How do you use destructuring, spread/rest, optional chaining, and nullish coalescing in everyday code?

THEORY

Destructuring pulls fields out of objects or arrays into variables, often with defaults. The **spread** operator copies or merges objects and arrays immutably, while **rest** collects remaining items into one variable. **Optional chaining** with question-mark-dot short-circuits to undefined instead of throwing on a missing path, and **nullish coalescing** with double question marks supplies a fallback only when the value is null or undefined.

STRONG ANSWER

These ES6+ features make day-to-day code cleaner and safer. I destructure props and hook returns constantly, and I use spread to update state immutably instead of mutating, which React relies on for re-renders. Optional chaining lets me safely read nested API fields that might be missing without a pile of and-checks. The important distinction I keep in mind is nullish coalescing versus the logical OR operator: double question marks only fall back on null or undefined, whereas OR also falls back on falsy values like 0 or empty string, which can cause subtle bugs. Picking the right one depends on whether 0 and empty string are valid values.

CODE · TypeScript

```
interface Props {
  title: string;
  count?: number;
  user?: { address?: { city: string } };
}

function Banner({ title, count = 0, user }: Props) {
  // optional chaining: no crash if address is missing
  const city = user?.address?.city ?? "Unknown";

  // nullish vs OR: 0 stays 0 with ??, but OR would drop it
  const shown = count ?? 0;

  return `${title} (${shown}) - ${city}`;
}

// spread to update immutably, rest to split
const base = { theme: "dark", fontSize: 14 };
const updated = { ...base, fontSize: 16 };

const [head, ...tail] = [1, 2, 3, 4]; // head=1, tail=[2,3,4]
```

Q19

TYPESCRIPT & JAVASCRIPT (ES6+)

How do map, filter, and reduce work, and why do they matter for immutability in React?

THEORY

map transforms each element into a new array of the same length, **filter** keeps only elements passing a predicate, and **reduce** folds an array down to a single accumulated value. Crucially, all three **return new arrays/values and never mutate the original**. That immutability is what lets React detect changes by reference and re-render correctly.

STRONG ANSWER

I use these constantly, especially in React Native where I render lists. The reason immutability matters is that React and state libraries compare by reference, so if I mutate an array in place with push or splice, the reference doesn't change and the UI may not update. map, filter, and reduce always give me a fresh array, so updating one item means mapping over state and replacing just that element. reduce is the most flexible, it can group, sum, or build a lookup map in one pass. The trade-off is that chaining many of them creates intermediate arrays, so for very large lists I might combine logic into a single reduce.

CODE · TypeScript

```
interface Todo { id: string; done: boolean; text: string; }

const todos: Todo[] = [
  { id: "1", done: false, text: "Buy milk" },
  { id: "2", done: true, text: "Walk dog" },
];

// map: immutable update of one item
const toggled = todos.map((t) =>
  t.id === "1" ? { ...t, done: true } : t
);

// filter: remove completed
const active = todos.filter((t) => !t.done);

// reduce: count remaining
const openCount = todos.reduce(
  (sum, t) => (t.done ? sum : sum + 1),
  0
);

console.log(active.length, openCount); // original todos untouched
```

ARCHITECTURE / FLOW

```
[a, b, c]
| map(fn)    -> [fn(a), fn(b), fn(c)]  (same length)
| filter(p)  -> [a, c]                  (subset)
| reduce(f,0)-> single value            (fold)

All return NEW data -> reference changes -> React re-renders
```

Q20

TYPESCRIPT & JAVASCRIPT (ES6+)

How would you migrate a live JavaScript React Native codebase to TypeScript without halting feature work?

THEORY

The realistic approach is **incremental migration, not a big-bang rewrite**. You enable TypeScript with `allowJs` so JS and TS coexist, rename files to `.ts/.tsx` file by file, and turn on strict mode gradually. Priority is given to **shared, high-risk code first**: API layers, models, and utilities, locking down **type-safe API contracts** so data shapes are trusted everywhere downstream.

STRONG ANSWER

I've done this incrementally so the team kept shipping features. First I add a `tsconfig` with `allowJs` and `checkJs` off, so existing JS keeps working and we migrate file by file rather than all at once. I start with the highest-leverage layer, the API client and data models, defining interfaces for every response so the typed contract flows into screens and reducers. I keep strict off initially to avoid thousands of errors, then enable flags one at a time, with `strictNullChecks` usually being the most valuable. I use any sparingly as a temporary bridge but track it with a lint rule and a no-implicit-any goal, and for runtime safety on API boundaries I'll add a validator like `zod` so the TypeScript types actually match what the server sends.

CODE · TypeScript

```
// tsconfig.json (phased strictness)
{
  "compilerOptions": {
    "allowJs": true,          // JS and TS coexist
    "checkJs": false,
    "strict": false,          // flip flags on gradually
    "strictNullChecks": true, // enable first - highest value
    "noImplicitAny": false,   // tighten later
    "jsx": "react-native"
  }
}

// type-safe API contract: define once, trust everywhere
export interface UserDTO {
  id: string;
  name: string;
  email: string | null;
}

export async function getUser(id: string): Promise<UserDTO> {
  const res = await fetch(`/users/${id}`);
  return (await res.json()) as UserDTO; // ideally validate with zod
}
```

ARCHITECTURE / FLOW

Big-bang rewrite -> BLOCKS features, high risk (avoid)

Incremental migration:

1. `tsconfig allowJs` (JS + TS coexist)
2. `types/` + API contracts (models first)
3. rename `.js` -> `.ts`, leaf modules outward
4. enable strict flags one at a time
 `strictNullChecks` -> `noImplicitAny` -> `strict`
5. `zod` validation at API boundary (runtime == types)

3

State Management & Data Fetching

Context, Redux Toolkit, thunks, selectors, React Query caching & mutations.

Q21–Q30

Q21

STATE MANAGEMENT & DATA FETCHING

What is the difference between local and global state, and when do you actually need a state management library?

THEORY

Local state lives inside a single component (via `useState` or `useReducer`) and is only needed by that component or its children. **Global state** is data many unrelated screens need, like the logged-in user, theme, or cart. You only reach for a library like Redux or Zustand when **prop drilling becomes painful** or when shared state changes frequently across the app.

STRONG ANSWER

Local state is anything that only one component cares about, like whether a modal is open or the text in an input field. Global state is shared data that lots of screens need, such as the authenticated user or app theme. Early on I keep everything local and only lift state up when a sibling needs it. I introduce a state library only when I find myself passing the same prop through three or four levels, or when many screens read and write the same data, because at that point Context or a store is cleaner than prop drilling.

CODE · TypeScript / TSX

```
// Local state - stays in the component
function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  // only this component cares about these values
  return <TextInput value={email} onChangeText={setEmail} />;
}

// Global state - many screens need the logged-in user
// Profile, Settings, Header all read auth.user
// -> a good candidate for Context, Redux or Zustand
```

ARCHITECTURE / FLOW

Local state:	Global state:
[Component]	[Store / Context]
<code>useState</code>	/ \
v	Header Cart Profile
(self only)	(shared across screens)

Q22

STATE MANAGEMENT & DATA FETCHING

How does the Context API work and what are its main limitations for state management?

THEORY

The **Context API** lets you pass data through the tree without prop drilling using a **Provider** and **useContext**. Its biggest limitation is **re-renders**: when the context value changes, every component that consumes that context re-renders, even if it only uses a small part of the value. It also has no built-in tooling for async logic, devtools, or memoized selectors, so it works best for **low-frequency data** like theme or auth, not fast-changing state.

STRONG ANSWER

Context gives you a **Provider** at the top and **useContext** anywhere below to read the value, which removes prop drilling for things like theme or the current user. The catch is that any change to the context value re-renders every consumer, regardless of which field they read, so it can cause performance issues with frequently updated state. There are also no selectors or middleware out of the box. So I use Context for slow-moving global data and reach for Redux Toolkit or Zustand when state updates often or when I need selectors and devtools.

CODE · TypeScript / TSX

```
type AuthCtx = { user: User | null; login: (u: User) => void };
const AuthContext = createContext<AuthCtx | null>(null);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  // value changes -> ALL consumers re-render, even theme-only ones
  const value = useMemo(() => ({ user, login: setUser }), [user]);
  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export const useAuth = () => {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be inside AuthProvider');
  return ctx;
};
```

ARCHITECTURE / FLOW

```
Provider value changes
  |
  v
+-----+-----+
v         v         v
ConsumerA ConsumerB ConsumerC
(all re-render, even if unused)
```

Q23

STATE MANAGEMENT & DATA FETCHING

Can you explain the core building blocks of Redux Toolkit: store, slice, reducers, and actions?

THEORY

Redux Toolkit (RTK) is the standard, modern way to write Redux with far less boilerplate. A **slice** bundles a piece of state with its reducers and auto-generates **action** creators. **Reducers** are pure functions that take state and an action and return the next state, and RTK lets you write 'mutating' logic safely using Immer. The **store** is the single source of truth that holds all slices combined.

STRONG ANSWER

Redux Toolkit is the official way to use Redux today and it removes most of the old boilerplate. I use createSlice to define a feature's state plus its reducers, and it automatically generates the matching action creators for me.

Reducers look like they mutate state, but Immer under the hood produces a new immutable state. Then configureStore combines all slices into one store, and components read with useSelector and update with dispatch.

CODE · TypeScript

```
import { createSlice, configureStore, PayloadAction } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1; }, // Immer
    addBy: (state, action: PayloadAction<number>) => {
      state.value += action.payload;
    },
  },
});

export const { increment, addBy } = counterSlice.actions;
export const store = configureStore({
  reducer: { counter: counterSlice.reducer },
});

export type RootState = ReturnType<typeof store.getState>;
```

ARCHITECTURE / FLOW

```

Component                Store
| dispatch(action)      |
+-----> reducer(state, action)
|
| new state
|
+----- useSelector (re-render)

```

Q24**STATE MANAGEMENT & DATA FETCHING**

How do you handle asynchronous logic like API calls in Redux Toolkit using createAsyncThunk?

THEORY

createAsyncThunk is RTK's helper for async flows like fetching data. It dispatches three lifecycle actions automatically: **pending**, **fulfilled**, and **rejected**. You handle these inside the slice's **extraReducers** to update loading, data, and error state. This keeps async logic out of components and gives you consistent loading and error handling.

STRONG ANSWER

createAsyncThunk wraps an async function and automatically dispatches pending, fulfilled, and rejected actions based on the promise. I handle those three states in extraReducers using the builder syntax, so I can set a loading flag on pending, store data on fulfilled, and capture the error on rejected. The component just dispatches the thunk and reads status from the store. This gives me predictable loading and error handling without writing the action plumbing myself.

CODE · TypeScript

```
import { createAsyncThunk, createSlice } from '@reduxjs/toolkit';

export const fetchUser = createAsyncThunk('user/fetch', async (id: string) => {
  const res = await fetch(`/api/users/${id}`);
  return (await res.json()) as User;
});

const userSlice = createSlice({
  name: 'user',
  initialState: { data: null as User | null, loading: false, error: '' },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUser.pending, (s) => { s.loading = true; s.error = ''; })
      .addCase(fetchUser.fulfilled, (s, a) => { s.loading = false; s.data = a.payload; })
      .addCase(fetchUser.rejected, (s, a) => {
        s.loading = false; s.error = a.error.message ?? 'Failed';
      });
  },
});
export default userSlice.reducer;
```

ARCHITECTURE / FLOW

```
dispatch(fetchUser(id))
  |
  v
[pending] -> loading = true
  |
  API call
 /    \
[fulfilled] [rejected]
data=...   error=...
```

Q25

STATE MANAGEMENT & DATA FETCHING

What are selectors and why is memoization important when using useSelector?

THEORY

A **selector** is a function that reads a specific piece of state from the store, keeping that knowledge in one place. With `useSelector`, the component re-renders whenever the selected value changes by reference, so returning a **new object or array each time** causes needless re-renders. **`createSelector`** from `Reselect` memoizes derived data so it only recomputes when its inputs actually change. This keeps expensive calculations and re-renders under control.

STRONG ANSWER

A selector is just a function that pulls a slice of state out of the store, which keeps the shape of the state encapsulated. The problem with `useSelector` is that it re-renders the component whenever the returned value changes by reference, so if I compute a filtered array inline it creates a new array every time and re-renders constantly. To fix that I use `createSelector` from `Reselect`, which memoizes the result and only recomputes when the inputs change. That way derived data like a filtered or sorted list stays stable and cheap.

CODE · TypeScript / TSX

```
import { createSelector } from '@reduxjs/toolkit';
import { useSelector } from 'react-redux';
import type { RootState } from './store';

const selectTodos = (s: RootState) => s.todos.items;
const selectFilter = (s: RootState) => s.todos.filter;
```

CODE · TypeScript / TSX (continued)

```
// memoized: recomputes only when items or filter change
export const selectVisibleTodos = createSelector(
  [selectTodos, selectFilter],
  (items, filter) => items.filter((t) => t.status === filter)
);

function TodoList() {
  // stable reference -> no re-render unless inputs change
  const todos = useSelector(selectVisibleTodos);
  return <FlatList data={todos} renderItem={({ item }) => <Row t={item} /> />;
}
```

ARCHITECTURE / FLOW

```
inputs change?
  no -> return cached result (no re-render)
  yes -> recompute -> cache -> return

selectItems --\
               > createSelector -> visibleTodos
selectFilter -/
```

Q26

STATE MANAGEMENT & DATA FETCHING

What is React Query (TanStack Query) and how does server state differ from client state?

THEORY

React Query manages **server state**, which is data that lives on a backend and you only borrow, like a list of products fetched from an API. This differs from **client state**, which your app fully owns, like a toggle or form input. Server state is asynchronous, can become stale, and may be shared by many users, so it needs caching, background refetching, and synchronization that tools like Redux were never designed for. React Query handles fetching, caching, and updating that remote data for you.

STRONG ANSWER

React Query is a library specifically for server state, meaning data that actually belongs to your backend and that you fetch and cache on the client. The key insight is that server state is fundamentally different from client state: it's async, it can go stale because someone else changed it, and you don't truly own it. Redux is great for client state like UI toggles, but it forces you to hand-write loading, caching, and refetching for server data. React Query gives you that out of the box with `useQuery`, so I use it for anything that comes from an API and keep Redux or Zustand for pure client state.

CODE · TypeScript / TSX

```
import { useQuery } from '@tanstack/react-query';

async function getProducts(): Promise<Product[]> {
  const res = await fetch('/api/products');
  if (!res.ok) throw new Error('Network error');
  return res.json();
}

function ProductList() {
  const { data, isLoading, isError, refetch } = useQuery({
    queryKey: ['products'],
    queryFn: getProducts,
  });

  if (isLoading) return <ActivityIndicator />;
  if (isError) return <Button title="Retry" onPress={() => refetch()} />;
```

CODE · TypeScript / TSX (continued)

```
return <FlatList data={data} renderItem={({ item }) => <Card p={item} /> } />;
}
```

ARCHITECTURE / FLOW

Client state (you own it)	Server state (backend owns it)
- UI toggles	- API lists
- form inputs	- user records
-> Redux/Zustand useState	-> React Query (cache + refetch)

Q27

STATE MANAGEMENT & DATA FETCHING

How do caching, staleTime, refetching, and query invalidation work in React Query?

THEORY

React Query caches each query by its **queryKey** and serves the cached data instantly while deciding whether to refetch. **staleTime** controls how long data is considered fresh; while fresh, no background refetch happens, and after it goes stale React Query refetches on triggers like refocus or remount. **gcTime** (cacheTime) controls how long unused data stays in memory. **Invalidation** with `queryClient.invalidateQueries` marks queries stale so they refetch, which is how you refresh data after a change.

STRONG ANSWER

Every query is cached under its `queryKey`, so navigating back to a screen shows cached data immediately instead of a spinner. `staleTime` decides how long that cached data is treated as fresh; within that window React Query won't refetch, and once it's stale it refetches in the background on events like window or app refocus. `gcTime` is how long inactive data is kept before garbage collection. When I do something that changes the server, like creating an item, I call `invalidateQueries` on the related key so React Query knows to refetch and the UI stays in sync.

CODE · TypeScript / TSX

```
import { useQuery, useQueryClient } from '@tanstack/react-query';

function useProducts() {
  return useQuery({
    queryKey: ['products'],
    queryFn: getProducts,
    staleTime: 1000 * 60, // fresh for 60s, no refetch in that window
    gcTime: 1000 * 60 * 5, // kept 5 min after unused
  });
}

function RefreshButton() {
  const qc = useQueryClient();
  // mark stale -> triggers a refetch of products
  return <Button title="Refresh" onPress={() =>
    qc.invalidateQueries({ queryKey: ['products'] }) } />;
}
```

ARCHITECTURE / FLOW

```
fetch -> [fresh] --staleTime--> [stale]
      |                     |
      |                     |
  serve cache, no refetch  refetch on
                          focus/remount/invalidate

unused -> wait gcTime -> removed from cache
```

Q28

STATE MANAGEMENT & DATA FETCHING

How do you perform mutations and implement optimistic updates with React Query?

THEORY

A **mutation** is any operation that changes server data, like create, update, or delete, handled with **useMutation**. **Optimistic updates** mean you update the UI immediately as if the request already succeeded, then roll back if it fails. You do this in `onMutate` by cancelling in-flight queries, snapshotting the cache, and writing the expected value, then restoring the snapshot in `onError`. Finally `onSettled` invalidates the query to sync with the server's real state.

STRONG ANSWER

For anything that writes to the server I use `useMutation`, which gives me `mutate`, plus `isPending` and error state. Optimistic updates make the app feel instant: in `onMutate` I cancel ongoing queries, save a snapshot of the current cache, and immediately apply the expected change. If the request fails, `onError` rolls back to that snapshot so the user never sees broken data. Then in `onSettled` I invalidate the query so the cache reconciles with what the server actually saved.

CODE · TypeScript / TSX

```
const qc = useQueryClient();
const addTodo = useMutation({
  mutationFn: (text: string) => api.post('/todos', { text }),
  onMutate: async (text) => {
    await qc.cancelQueries({ queryKey: ['todos'] });
    const prev = qc.getQueryData<Todo[]>(['todos']);
    qc.setQueryData<Todo[]>(['todos'], (old = []) => [
      ...old, { id: 'temp', text, done: false },
    ]);
    return { prev }; // context for rollback
  },
  onError: (_, _text, ctx) => qc.setQueryData(['todos'], ctx?.prev),
  onSettled: () => qc.invalidateQueries({ queryKey: ['todos'] }),
});
```

ARCHITECTURE / FLOW

```
onMutate: cancel + snapshot + apply optimistic UI
      |
      +-- send request
      /      \
  success    failure
      |          |
onSettled   onError: restore snapshot
invalidate -> refetch
```

Q29

STATE MANAGEMENT & DATA FETCHING

When would you choose Redux Toolkit versus React Query versus Zustand?

THEORY

These tools solve different problems. **React Query** is for **server state**, the async data fetched from APIs that needs caching and refetching. **Zustand** is a tiny, simple store for **client state** with minimal boilerplate and selector-based subscriptions. **Redux Toolkit** is also for client state but shines in **large apps** that need strict structure, middleware, devtools, and predictable patterns across a big team. In practice many apps pair React Query for server data with Zustand or Redux for client state.

STRONG ANSWER

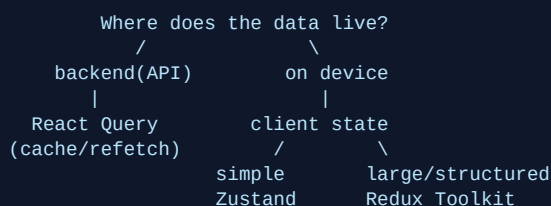
I think of it as server state versus client state. React Query owns server state, so anything coming from an API I handle there for caching and refetching. For client state I pick between Zustand and Redux Toolkit: Zustand when I want something lightweight with very little boilerplate, and Redux Toolkit when the app is large, the team is big, or I need strong conventions, middleware, and devtools. A very common real-world setup is React Query for all the API data plus Zustand for a bit of UI and auth state, which keeps each tool doing what it's best at.

CODE · TypeScript / TSX

```
// Server state -> React Query
const { data } = useQuery({ queryKey: ['cart'], queryFn: getCart });

// Client state, lightweight -> Zustand
import { create } from 'zustand';
type UI = { theme: 'light' | 'dark'; toggle: () => void };
export const useUI = create<UI>((set) => ({
  theme: 'light',
  toggle: () => set((s) => ({ theme: s.theme === 'light' ? 'dark' : 'light' })),
}));
const theme = useUI((s) => s.theme); // selector subscription

// Client state, large structured app -> Redux Toolkit
// configureStore + slices + middleware + devtools
```

ARCHITECTURE / FLOW**Q30****STATE MANAGEMENT & DATA FETCHING**

What does it mean to normalize state, and how does it help avoid unnecessary re-renders?

THEORY

Normalizing state means storing data in a flat, lookup-friendly shape, typically an object keyed by id plus an array of ids, instead of deeply nested arrays. This avoids duplication, makes updates to a single item cheap, and prevents touching unrelated parts of state. RTK provides **createEntityAdapter** to do this for you. Combined with **memoized selectors**, components can subscribe to a single entity by id so only that row re-renders when it changes.

STRONG ANSWER

Normalizing means I store collections as a dictionary of items keyed by id with a separate list of ids, rather than a big nested array. The benefit is that updating one item only changes that entry, so I don't replace the whole array and trigger every row to re-render. Redux Toolkit's **createEntityAdapter** gives me this structure plus ready-made reducers and selectors. Paired with per-id selectors, a list row can subscribe only to its own item, so editing one todo re-renders just that row instead of the entire list.

CODE · TypeScript / TSX

```
import { createEntityAdapter, createSlice } from '@reduxjs/toolkit';

const todosAdapter = createEntityAdapter<Todo>();
// state shape: { ids: [...], entities: { [id]: Todo } }

const todosSlice = createSlice({
  name: 'todos',
  initialState: todosAdapter.getInitialState(),
  reducers: {
    addOne: todosAdapter.addOne,
    updateOne: todosAdapter.updateOne, // touches only that entity
  },
});

export const { selectById, selectIds } =
  todosAdapter.getSelectors((s: RootState) => s.todos);

// Row subscribes to ONE entity -> only it re-renders on change
function TodoRow({ id }: { id: string }) {
  const todo = useSelector((s: RootState) => selectById(s, id));
  return <Text>{todo?.text}</Text>;
}
```

ARCHITECTURE / FLOW

Nested (bad):	Normalized (good):
[{id:1,..},	ids: [1, 2, 3]
{id:2,..}, -->	entities: {
{id:3,..}]	1:{..}, 2:{..}, 3:{..}
	}
update -> new array	update id=2 -> only row 2
(all rows re-render)	re-renders

4

Animations, Gestures & Performance

Threads, Reanimated worklets, gestures, 60fps, memoization & profiling.

Q31–Q40

Q31

ANIMATIONS, GESTURES & PERFORMANCE

What is the difference between the JS thread and the UI thread in React Native, and why do animations sometimes jank?

THEORY

React Native runs your JavaScript logic on the **JS thread** while the native rendering and gesture work happens on the **UI thread (main thread)**. With the old Animated API, animation values are computed in JS and sent across the bridge, so if the **JS thread is busy** (data fetching, heavy state updates, big list renders) it can't push updates in time and frames drop. **Jank** happens because the screen needs a new frame every 16ms but the blocked JS thread misses that window.

STRONG ANSWER

React Native has two main threads: the JS thread where my application code and React run, and the UI thread which actually draws to the screen and handles touch. When an animation is driven from JavaScript, every frame's value has to be calculated on the JS thread and communicated to native, so if the JS thread is busy doing something like a network response handler or a large re-render, those animation updates arrive late and the user sees stutter. That's the classic jank problem. The fix is either to use the native driver so the animation runs without the JS thread, or to use Reanimated, which runs the animation logic directly on the UI thread via worklets. Keeping heavy work off the JS thread is the underlying principle.

CODE · TypeScript / TSX

```
import { Animated, Easing } from 'react-native';
import { useRef, useEffect } from 'react';

export function FadeIn() {
  const opacity = useRef(new Animated.Value(0)).current;

  useEffect(() => {
    Animated.timing(opacity, {
      toValue: 1,
      duration: 300,
      easing: Easing.out(Easing.ease),
      // runs on UI thread, immune to JS thread jank
      useNativeDriver: true,
    }).start();
  }, [opacity]);

  return <Animated.View style={{ opacity }} />;
}
```

ARCHITECTURE / FLOW

JS THREAD	UI THREAD
-----	-----
app logic, React render	layout + draw
network handlers	touch handling
state updates	native animation
	^
busy? updates arrive late	----> DROPPED FRAME
v	
useNativeDriver / Reanimated	-----+ (bypasses JS)

Q32

ANIMATIONS, GESTURES & PERFORMANCE

What is the difference between the Animated API and Reanimated, and when would you choose each?

THEORY

The built-in **Animated API** ships with React Native and works well for simple animations, especially with **useNativeDriver: true**, but it can only natively drive a limited set of non-layout properties like opacity and transform. **Reanimated** runs animation logic on the UI thread using **worklets**, supports layout animations, gestures, and complex interactive choreography with far less jank. For basic fades or slides Animated is fine; for gesture-driven or performance-critical UI, Reanimated is the modern standard.

STRONG ANSWER

Animated is the API that comes with React Native out of the box. It's good enough for simple things, and with `useNativeDriver` it offloads opacity and transforms to native, but it can't natively animate layout properties like width or height, and complex interactive animations get awkward. Reanimated is a third-party library that runs almost everything on the UI thread through worklets, so even gesture-driven animations stay at 60fps without crossing the bridge each frame. It also has a much cleaner hooks-based API with shared values and `useAnimatedStyle`, plus built-in layout animations. In my projects I reach for Animated for trivial one-off animations and Reanimated whenever there's a gesture or anything that needs to feel buttery.

CODE · TypeScript / TSX

```
// Reanimated equivalent of a fade-in
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withTiming,
} from 'react-native-reanimated';
import { useEffect } from 'react';

export function FadeIn() {
  const opacity = useSharedValue(0);

  useEffect(() => {
    opacity.value = withTiming(1, { duration: 300 });
  }, [opacity]);

  const style = useAnimatedStyle(() => ({ opacity: opacity.value }));

  return <Animated.View style={style} />;
}
```

Q33

ANIMATIONS, GESTURES & PERFORMANCE

What are shared values and useAnimatedStyle in Reanimated, and how do they work together?

THEORY

A **shared value** (`useSharedValue`) is a mutable container whose `.value` can be read and written on both the JS and UI threads, and it is the source of truth for an animation. **useAnimatedStyle** is a worklet that reads shared values and returns a style object, re-running on the **UI thread** whenever those values change. Because updates happen on the UI thread, changing a shared value does **not trigger a React re-render**, which is what keeps animations smooth.

STRONG ANSWER

A shared value is like a special piece of state that both threads can access, created with `useSharedValue`, and you mutate it through its `.value` property. `useAnimatedStyle` is a hook that takes a worklet returning a style; `Reanimated` tracks which shared values that worklet reads, and whenever they change it recomputes the style directly on the UI thread. The key benefit is that updating a shared value doesn't cause a React component re-render, so I can animate sixty times a second without React doing any work. I typically declare shared values for things like translation or scale, then map them to transforms inside `useAnimatedStyle`. It's the foundation of basically every `Reanimated` animation I write.

CODE · TypeScript / TSX

```
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withSpring,
} from 'react-native-reanimated';
import { Button, View } from 'react-native';

export function ScaleBox() {
  const scale = useSharedValue(1);

  const style = useAnimatedStyle(() => ({
    transform: [{ scale: scale.value }],
  }));

  return (
    <View>
      <Animated.View style={[{ width: 80, height: 80 }, style]} />
      <Button title="Grow" onPress={() => (scale.value = withSpring(1.5))} />
    </View>
  );
}
```

ARCHITECTURE / FLOW

```
scale (shared value) ---> useAnimatedStyle (worklet, UI thread)
|                               |
|                               v
.value = withSpring(1.5)      returns { transform:[{scale}] }
|                               |
+--- NO React re-render ---> View updates on UI thread
```

Q34**ANIMATIONS, GESTURES & PERFORMANCE****What is a worklet in Reanimated, and what are `runOnUI` and `runOnJS` used for?****THEORY**

A **worklet** is a small JavaScript function that `Reanimated` can serialize and run on the **UI thread** in a separate JS context, which is how animation logic stays off the main JS thread. **`runOnUI`** schedules a function to execute on the UI thread, while **`runOnJS`** lets a worklet call back into a regular JS-thread function (for example to update React state or navigate). You need `runOnJS` because UI-thread worklets can't directly touch React state or most JS-thread APIs.

STRONG ANSWER

A worklet is a function marked to run on the UI thread; many Reanimated hooks like `useAnimatedStyle` and gesture callbacks are worklets automatically. Because that code runs in its own isolated context on the UI thread, it can't directly call normal JS-thread things like `setState`. So when I'm inside a worklet and need to update React state or trigger navigation, I wrap that call in `runOnJS`, which marshals it back to the JS thread. Going the other direction, `runOnUI` lets me explicitly kick off a worklet from the JS thread. The mental model I keep is: worklets and shared values live on the UI thread, React state lives on the JS thread, and `runOnJS` is the bridge between them.

CODE · TypeScript / TSX

```
import { Gesture } from 'react-native-gesture-handler';
import { useSharedValue, runOnJS } from 'react-native-reanimated';
import { useState } from 'react';

export function useTapCounter() {
  const [count, setCount] = useState(0);
  const pressed = useSharedValue(false);

  const tap = Gesture.Tap().onEnd(() => {
    'worklet';
    pressed.value = true; // UI-thread safe
    // setCount is a JS-thread fn, must hop back
    runOnJS(setCount)((c: number) => c + 1);
  });

  return { tap, count };
}
```

ARCHITECTURE / FLOW

JS THREAD		UI THREAD
-----		-----
React state (setCount)		worklets, shared values
^		
runOnJS(setCount)	<-----+ (worklet calls back)	^
+----- runOnUI(fn)	-----+ (start a worklet)	

Q35**ANIMATIONS, GESTURES & PERFORMANCE**

How do you handle gestures with react-native-gesture-handler, for example a pan gesture?

THEORY

react-native-gesture-handler replaces React Native's default touch system with native gesture recognizers that run on the **UI thread**, so gestures stay responsive even when the JS thread is busy. You build gestures with the **Gesture** API (`Gesture.Pan`, `Gesture.Tap`, `Gesture.Pinch`) and attach them through a **GestureDetector**. Combined with Reanimated shared values, gesture callbacks are worklets that update animation state directly on the UI thread.

STRONG ANSWER

Gesture Handler gives me native, declarative gesture recognizers instead of relying on the JS-based responder system. I create a gesture like `Gesture.Pan()`, chain handlers such as `onUpdate` and `onEnd`, and wrap my view in a `GestureDetector`. Inside those callbacks, which are worklets, I update Reanimated shared values from the event translation, and a `useAnimatedStyle` maps them to transforms, so dragging feels instant. Because the recognition happens natively on the UI thread, the drag keeps tracking my finger even if JS is doing other work. I also like that you can compose gestures, like `Race` or `Simultaneous`, for more complex interactions. This pairing of Gesture Handler and Reanimated is my default for any draggable UI.

CODE · TypeScript / TSX

```
import { Gesture, GestureDetector } from 'react-native-gesture-handler';
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withSpring,
} from 'react-native-reanimated';

export function Draggable() {
  const x = useSharedValue(0);
  const y = useSharedValue(0);

  const pan = Gesture.Pan()
    .onUpdate((e) => {
      x.value = e.translationX;
      y.value = e.translationY;
    })
    .onEnd(() => {
      x.value = withSpring(0);
      y.value = withSpring(0);
    });

  const style = useAnimatedStyle(() => ({
    transform: [{ translateX: x.value }, { translateY: y.value }],
  }));

  return (
    <GestureDetector gesture={pan}>
      <Animated.View style={[{ width: 100, height: 100 }, style]} />
    </GestureDetector>
  );
}
```

Q36

ANIMATIONS, GESTURES & PERFORMANCE

What does it mean to achieve 60fps in React Native, and what commonly causes dropped frames?

THEORY

Most screens refresh 60 times per second, giving each frame a budget of about **16ms** (roughly 8ms on 120Hz displays) to compute and render. If either the JS thread or the UI thread takes longer than that budget, the frame is **dropped** and the user perceives stutter. Common causes include heavy synchronous work on the JS thread, large or unoptimized list rendering, expensive re-renders, and animating layout properties from JS instead of natively.

STRONG ANSWER

Hitting 60fps means delivering a fresh frame every 16 milliseconds, so anything that blocks a thread past that window causes a dropped frame. On the JS side the usual culprits are big synchronous computations, oversized state updates that re-render large trees, or chatty bridge traffic from JS-driven animations. On the UI side it's usually deep view hierarchies and expensive layout. My go-to remedies are moving animations to the native driver or Reanimated, memoizing components and callbacks to cut re-renders, virtualizing long lists, and deferring non-urgent work with InteractionManager or after interactions. Essentially the goal is to keep both threads doing as little as possible within each frame.

CODE · TypeScript / TSX

```
import { InteractionManager } from 'react-native';
import { useEffect, useState } from 'react';

// Defer heavy work until after animations/interactions finish,
// protecting the 16ms-per-frame budget during the transition.
```

CODE · TypeScript / TSX (continued)

```
export function useDeferredData(load: () => Promise<unknown>) {
  const [data, setData] = useState<unknown>(null);

  useEffect(() => {
    const task = InteractionManager.runAfterInteractions(async () => {
      const result = await load();
      setData(result);
    });
    return () => task.cancel();
  }, [load]);

  return data;
}
```

ARCHITECTURE / FLOW

Frame budget at 60fps = ~16.6ms per frame

```
|<----- 16.6ms ----->|<----- 16.6ms ----->|
[ JS + UI work fits ]   [ work fits ]           OK = smooth

|<----- 16.6ms ----->|<----- 16.6ms ----->|
[ work overruns.....|.....] X DROPPED         jank/stutter
```

Q37

ANIMATIONS, GESTURES & PERFORMANCE

How do you prevent unnecessary re-renders in React Native using React.memo, useCallback, and useMemo?

THEORY

React re-renders a component whenever its state or props change, and a parent re-render cascades to children even if their props are identical. **React.memo** skips re-rendering a child when its props are shallow-equal, **useCallback** memoizes a function so its reference stays stable between renders, and **useMemo** caches an expensive computed value. Used together they stop reference churn from triggering wasted renders, which is critical for list items and animation-heavy screens.

STRONG ANSWER

The core problem is that when a parent re-renders, all its children re-render by default, and passing fresh inline functions or objects each render defeats any optimization. I wrap pure child components in **React.memo** so they only re-render when their actual props change. To keep that effective, I use **useCallback** for handlers I pass down so the function reference is stable, and **useMemo** for derived data or objects that are expensive to compute or that would otherwise be a new reference every render. I'm careful not to over-apply these, since memoization has its own cost and only pays off when re-renders are actually expensive or frequent, like in list rows. Profiling first tells me where it matters.

CODE · TypeScript / TSX

```
import React, { useCallback, useMemo, useState } from 'react';
import { FlatList } from 'react-native';

const Row = React.memo(function Row({ item, onPress }: any) {
  return <Item title={item.title} onPress={onPress} />;
});

export function List({ data }: { data: any[] }) {
  const [, force] = useState(0);

  // stable reference -> Row's React.memo stays effective
```

CODE · TypeScript / TSX (continued)

```
const onPress = useCallback((id: string) => force((n) => n + 1), []);

// cached unless data changes
const sorted = useMemo(() => [...data].sort(), [data]);

return (
  <FlatList
    data={sorted}
    renderItem={({ item }) => <Row item={item} onPress={onPress} />}
    keyExtractor={(i) => i.id}
  />
);
}
```

Q38

ANIMATIONS, GESTURES & PERFORMANCE

How do you tune FlatList performance with props like getItemLayout, keyExtractor, windowSize, and removeClippedSubviews?

THEORY

FlatList is virtualized, rendering only items near the viewport, but it still has to measure, key, and recycle rows efficiently. **getItemLayout** lets it skip measurement for fixed-height rows so scrolling and scroll-to-index are instant, **keyExtractor** gives stable identity to avoid remounting, **windowSize** controls how many screens of content are kept rendered, and **removeClippedSubviews** detaches off-screen views to save memory. Pairing these with memoized row components keeps long lists smooth.

STRONG ANSWER

FlatList already virtualizes, but for long lists I tune a few props. If my rows are a fixed height I provide **getItemLayout**, which lets the list skip measuring and jump to any index immediately, eliminating a lot of layout work. I always supply a stable **keyExtractor** so items aren't unnecessarily remounted as data shifts. I adjust **windowSize** to balance how much off-screen content stays mounted against memory and blank-area-on-fast-scroll, and I enable **removeClippedSubviews** to unmount views that scroll out of view on Android especially. I also memoize the **renderItem** component and use **initialNumToRender** and **maxToRenderPerBatch** to control startup cost. The combination keeps even thousands of rows scrolling at 60fps.

CODE · TypeScript / TSX

```
import { FlatList } from 'react-native';
import React from 'react';

const ROW_HEIGHT = 72;
const Row = React.memo(({ item }: any) => <Item data={item} />);

export function BigList({ data }: { data: any[] }) {
  return (
    <FlatList
      data={data}
      renderItem={({ item }) => <Row item={item} />}
      keyExtractor={(i) => i.id}
      getItemLayout={(_, index) => ({
        length: ROW_HEIGHT,
        offset: ROW_HEIGHT * index,
        index,
      })}
      windowSize={7}
      initialNumToRender={10}
      maxToRenderPerBatch={10}
      removeClippedSubviews
    />
  );
}
```

CODE · TypeScript / TSX (continued)

```

    );
  }

```

ARCHITECTURE / FLOW

```

FlatList window (windowSize controls how many screens kept)

[ rendered above ]   <- recycled/clipped
+-----+
| VISIBLE VIEWPORT | <- rendered
+-----+
[ rendered below ]   <- recycled/clipped

getItemLayout(index) -> offset = ROW_HEIGHT * index (no measure)

```

Q39

ANIMATIONS, GESTURES & PERFORMANCE

What is the Hermes engine and what does it improve in a React Native app?

THEORY

Hermes is a lightweight JavaScript engine built by Meta specifically for React Native, and it is the **default engine** on modern versions. It compiles JavaScript to **bytecode ahead of time** during the build, so the app skips parsing and compiling JS at runtime. This mainly improves **startup time (TTI)**, reduces **memory usage**, and shrinks app size, with better tooling for profiling and debugging compared to JSC.

STRONG ANSWER

Hermes is the JavaScript engine Meta built and optimized for React Native, and it's the default now. Its big trick is ahead-of-time compilation: instead of shipping plain JS that gets parsed and compiled on the device at launch, the bundle is precompiled to bytecode at build time, so the engine just loads and runs it. That cuts time-to-interactive noticeably, lowers memory consumption, and reduces the download size of the app. It also integrates well with profiling tools so I can capture CPU samples and inspect performance. In practice enabling Hermes was one of the easiest wins for startup performance on the apps I've worked on.

CODE · TypeScript / TSX

```

// android/gradle.properties
// hermesEnabled=true   (default on modern RN)

// ios/Podfile
// :hermes_enabled => true

// Confirm Hermes is active at runtime:
export function isHermes(): boolean {
  // HermesInternal is injected globally only when Hermes runs
  return !!(global as any).HermesInternal;
}

// Usage
if (isHermes()) {
  console.log('Running on Hermes (bytecode, faster startup)');
}

```

ARCHITECTURE / FLOW

```

WITHOUT Hermes (JSC):
  ship JS source -> device parses + compiles -> runs   (slow start)

WITH Hermes:

```

ARCHITECTURE / FLOW (continued)

```
build: JS -> bytecode ===> device loads bytecode -> runs
                                   (faster TTI, less memory)
```

Q40

ANIMATIONS, GESTURES & PERFORMANCE

How do you profile and measure performance in a React Native app?**THEORY**

You can't optimize what you don't measure, so the first step is to capture real data. The on-device **Perf Monitor** shows live JS and UI thread frame rates, **React Native DevTools** (and the React Profiler) reveal component render counts and timings, and tools like **Flipper** or Hermes sampling profilers capture CPU traces and bridge traffic. The goal is to find which thread is overloaded and which components or operations are the bottleneck before changing code.

STRONG ANSWER

My approach is always measure first, then optimize. I start with the in-app Perf Monitor to see whether the JS thread or the UI thread is dropping frames, which immediately tells me where to look. For render issues I use React Native DevTools or the React Profiler to see which components render too often and how long they take, which usually points me to missing memoization. For deeper CPU work I capture a sampling profile from Hermes or use Flipper to inspect performance and bridge activity. I also lean on tools that flag specific problems, like the highlight-updates feature to literally see what's re-rendering. Once I've identified the actual bottleneck, I apply the targeted fix and re-measure to confirm the improvement.

CODE · TypeScript / TSX

```
import React, { Profiler } from 'react';

// React's built-in Profiler logs render timing programmatically.
function onRender(
  id: string,
  phase: 'mount' | 'update',
  actualDuration: number,
) {
  if (actualDuration > 16) {
    console.warn(`[perf] ${id} ${phase} took ${actualDuration.toFixed(1)}ms`);
  }
}

export function ProfiledScreen({ children }: { children: React.ReactNode }) {
  return (
    <Profiler id="Screen" onRender={onRender}>
      {children}
    </Profiler>
  );
}

// Also: shake device -> 'Show Perf Monitor' for live JS/UI fps.
```

ARCHITECTURE / FLOW

MEASURE	->	IDENTIFY	->	FIX	->	RE-MEASURE
-----		-----		---		-----
Perf Monitor		which thread?		memo / native		confirm fps
RN DevTools		which component?		driver / list		restored
Flipper/CPU		what bridge work?		tuning		

5

Native Modules & the New Architecture

Bridge vs JSI, TurboModules, Fabric, Codegen and Swift native modules.

Q41–Q50

Q41

NATIVE MODULES & THE NEW ARCHITECTURE

What is a native module in React Native, and when would you actually need to write one?

THEORY

A **native module** is a piece of platform code (Swift, Objective-C, Kotlin, Java) exposed to JavaScript so your JS can call capabilities React Native doesn't ship out of the box. You reach for one when a feature needs a **platform API or SDK** that has no JS equivalent, when you need **performance** beyond what JS can give, or when you must **reuse existing native code**.

STRONG ANSWER

A native module is basically a bridge from JavaScript to platform code, so you can call into iOS or Android APIs that React Native doesn't expose by default. In my last project I wrote a custom audio module in Swift because we needed precise control over AVAudioSession and low-latency playback, and there was no community package that did exactly what we wanted. The rule of thumb I use is: if the feature requires a native SDK, a sensor, or performance-critical work, write a native module; otherwise stay in JS. I also check npm first, because most common needs like camera or storage already have well-maintained modules.

CODE · TypeScript

```
// JS side calling a native module
import { NativeModules } from 'react-native';

const { AudioModule } = NativeModules;

// AudioModule is implemented natively in Swift.
// Here we just call it like any JS object.
export async function playClip(path: string) {
  try {
    const durationMs = await AudioModule.play(path);
    console.log('Playing, duration:', durationMs);
  } catch (e) {
    console.warn('Native playback failed', e);
  }
}
```

ARCHITECTURE / FLOW

JavaScript		Native (Swift / Kotlin)
-----		-----
AudioModule.play(path)	--->	play() { AVAudioPlayer... }
^		
	result / error	
+-----+		

Q42

NATIVE MODULES & THE NEW ARCHITECTURE

Can you describe the old bridge architecture and what its main bottlenecks were?

THEORY

In the legacy architecture, the JS thread and native threads were fully separate and communicated only through an **asynchronous bridge**. Every call had to be **serialized to JSON**, queued, and sent across, which made the bridge a throughput choke point. The two big problems were the **serialization cost** of converting data to and from JSON, and the **forced async** nature meaning JS could never call native synchronously.

STRONG ANSWER

The old bridge kept JavaScript and native running on different threads that never shared memory, so they talked by passing serialized JSON messages back and forth over a single asynchronous channel. That design was fine for small payloads but became a bottleneck in two ways: every message had to be serialized and deserialized, which is expensive for large or frequent data like scroll events or big lists, and because the bridge was strictly asynchronous and batched, you got latency and frame drops when traffic spiked. A classic symptom was janky animations driven from JS, because gesture data piled up in the bridge queue. The New Architecture with JSI removes that serialization layer, which is the whole motivation behind it.

CODE · TypeScript

```
// Conceptual: every call becomes a serialized message.
// JS -> bridge -> native (and back), all async + JSON.

NativeModules.AudioModule.play('/clip.mp3');
// Internally roughly:
// 1. arguments serialized to JSON
// 2. message pushed onto the bridge queue (batched)
// 3. native thread dequeues, parses JSON
// 4. native runs play(), serializes result
// 5. result travels back async to JS

// You could NOT do this on the old bridge:
// const x = NativeModules.AudioModule.playSync(); // no sync return
```

ARCHITECTURE / FLOW

JS thread	BRIDGE (async, JSON)	Native thread
call() -->	+-----+ [msg][msg][msg] queue --> parse + run serialize / deserialize	
result <--	+-----+ [msg][msg][msg] queue <-- serialize	
bottleneck: JSON cost + batched async hops		

Q43

NATIVE MODULES & THE NEW ARCHITECTURE

What is JSI (the JavaScript Interface), explained as simply as you can?

THEORY

JSI is a lightweight C++ layer that lets JavaScript hold direct references to native objects and call their methods **without serializing to JSON**. Instead of message passing, native code installs **host objects/functions** directly into the JS runtime, so a JS call becomes essentially a regular function call into C++. This enables **synchronous calls** and removes the bridge as a bottleneck.

STRONG ANSWER

JSI is the foundation of the New Architecture. It's a thin C++ interface that lets the JavaScript engine talk directly to native code instead of sending JSON across a bridge. The key idea is that native code can expose host objects that live inside the JS runtime, so calling a native function from JS is like calling any other JS function, no serialization required. Because of that direct binding, calls can be synchronous and you can even pass references to native objects, which was impossible before. Practically, it's what makes TurboModules and Fabric fast, since both are built on top of JSI.

CODE · Swift

```
// Conceptual C++ JSI: install a host function into JS runtime.
#include <jsi/jsi.h>
using namespace facebook::jsi;

void install(Runtime& rt) {
    auto multiply = Function::createFromHostFunction(
        rt,
        PropNameID::forAscii(rt, "multiply"),
        2,
        [](Runtime& rt, const Value&, const Value* args, size_t) -> Value {
            double a = args[0].asNumber();
            double b = args[1].asNumber();
            return Value(a * b); // direct return, no JSON
        });
    // Now JS can call global.multiply(6, 7) synchronously.
    rt.global().setProperty(rt, "multiply", multiply);
}
```

ARCHITECTURE / FLOW

OLD bridge: JS --JSON--> queue --JSON--> Native (async)

JSI: JS ===direct C++ call=== Native
(host objects live inside the JS runtime,
no serialization, can be synchronous)

Q44**NATIVE MODULES & THE NEW ARCHITECTURE****How do TurboModules differ from the old-style native modules?****THEORY**

TurboModules are native modules built on **JSI** instead of the async bridge, so JS calls into them are direct C++ calls with no JSON serialization. They are also **lazily loaded**, meaning a module is initialized only when JS first uses it, instead of all modules loading at startup. They use **Codegen** to produce type-safe interfaces from a TypeScript spec.

STRONG ANSWER

Old native modules registered themselves with the bridge and were all eagerly loaded at app startup, and every call went through JSON serialization. TurboModules sit on top of JSI, so calls are direct and can be synchronous, and they only get instantiated the first time JavaScript actually references them, which improves startup time. Another big difference is type safety: you declare a TypeScript spec and Codegen generates the native interfaces, so the JS and native sides can't drift apart. In practice migrating a module means writing a spec file and conforming to the generated protocol; the business logic stays mostly the same.

CODE · TypeScript

```
// NativeAudioModule.ts (the TurboModule spec)
import type { TurboModule } from 'react-native';
import { TurboModuleRegistry } from 'react-native';

export interface Spec extends TurboModule {
  // Codegen reads these signatures to generate native protocols.
  play(path: string): Promise<number>;
  stop(): void;
  getVolume(): number; // can be synchronous on JSI
}

export default TurboModuleRegistry.getEnforcing<Spec>('AudioModule');
```

ARCHITECTURE / FLOW

```
Old module: eager load all + bridge (JSON, async)
TurboModule: lazy load on first use + JSI (direct, sync-capable)

JS -> TurboModuleRegistry.getEnforcing('AudioModule')
    -> JSI host object -> native Swift/Kotlin impl
```

Q45

NATIVE MODULES & THE NEW ARCHITECTURE

What is Fabric, and what problem does the new rendering system solve?

THEORY

Fabric is React Native's new rendering system, also built on JSI, that replaces the old UIManager bridge for creating and updating native views. It builds an immutable **shadow tree in C++** shared across threads and can apply layout and view updates **synchronously**, which improves consistency and responsiveness. It also enables better **interop**, concurrent React features, and reduced UI jank.

STRONG ANSWER

Fabric is the new rendering layer that handles turning your React component tree into actual native views. The old system sent view operations across the async bridge, which could cause tearing and dropped frames; Fabric instead keeps the shadow tree in C++ and uses JSI so it can commit updates more directly and even synchronously when needed. This gives more consistent layout, better integration with React's concurrent features like Suspense, and smoother UI under load. From a developer standpoint you mostly enable it via the New Architecture flag; the payoff is the rendering pipeline being faster and more predictable.

CODE · TypeScript / TSX

```
// You usually enable Fabric via the New Architecture flag,
// not by writing Fabric code directly.

// iOS: ios/Podfile.properties.json
{
  "newArchEnabled": "true"
}

// Android: android/gradle.properties
// newArchEnabled=true

// With Fabric on, your normal JSX renders through the new
// C++ shadow tree + synchronous commit pipeline automatically.
export function Card() {
  return <View style={{ padding: 16 }}><Text>Hello</Text></View>;
}
```

ARCHITECTURE / FLOW

```

React tree (JS)
  | JSI
  v
Shadow tree (C++, immutable, shared)
  | commit (can be synchronous)
  v
Mounting layer -> native UIView / android.view.View

Old UIManager: async bridge ops -> tearing / dropped frames

```

Q46**NATIVE MODULES & THE NEW ARCHITECTURE****What is Codegen, and how does it give you type-safe native specs?****THEORY**

Codegen is a build-time tool that reads your **TypeScript spec files** for TurboModules and Fabric components and generates the corresponding **native interfaces** (Objective-C++/Java/C++). This guarantees the JS contract and the native implementation **stay in sync**, since the native side must conform to generated protocols. It runs automatically during the build, so type mismatches surface as **compile errors** rather than runtime crashes.

STRONG ANSWER

Codegen is what makes the New Architecture type-safe. You write a spec as a TypeScript interface describing the methods and their parameter and return types, and Codegen generates the matching native boilerplate, like an Objective-C protocol on iOS and an abstract class on Android. Your native code then conforms to those generated types, so if you change a method signature in JS without updating native, the build fails instead of crashing at runtime. It runs as part of the normal build process, triggered by the codegenConfig in package.json. I really like it because it kills a whole class of bugs where the JS and native sides silently disagree on shapes.

CODE · TypeScript

```

// package.json - tells Codegen where the specs live.
{
  "codegenConfig": {
    "name": "AudioModuleSpec",
    "type": "modules",
    "jsSrcsDir": "src/specs",
    "ios": { "componentProvider": {} }
  }
}

// Codegen reads src/specs/NativeAudioModule.ts and emits, e.g.:
// - iOS: @protocol NativeAudioModuleSpec (Obj-C++)
// - Android: abstract class NativeAudioModuleSpec (Java)
// Your Swift/Kotlin code must conform, or the build fails.

```

ARCHITECTURE / FLOW

```

NativeAudioModule.ts (TS spec)
  |
  Codegen (build time)
  /      \
iOS protocol  Android abstract class
(Obj-C++)    (Java/Kotlin)
  \      /
native impl must conform -> mismatch = compile error

```

Q47

NATIVE MODULES & THE NEW ARCHITECTURE

How would you write a simple native module in Swift that exposes a method to JavaScript?

THEORY

On iOS you create a Swift class subclassing **NSObject** and expose it with the **@objc(Name)** annotation, plus a small Objective-C **bridging macro** (**RCT_EXTERN_MODULE**) so React Native can find it. Methods are exposed with **RCT_EXTERN_METHOD**, and you typically return results via a **resolver/rejecter** (promise) or a callback. The Swift method names and parameter labels must match the macro declarations exactly.

STRONG ANSWER

On iOS I make a Swift class that inherits from **NSObject** and annotate it with **@objc** plus the module name, then mark each exposed method with **@objc** so it's visible to the Objective-C runtime. Because RN's macros are Objective-C, I add a small **.m** file using **RCT_EXTERN_MODULE** and **RCT_EXTERN_METHOD** to register the class and its methods. For the audio module I exposed a play method that took a path and a resolver and rejecter, so the JS side could await it as a promise. The main gotchas are keeping the selector signatures identical between the **.m** macros and the Swift method, and remembering to implement **requiresMainQueueSetup**.

CODE · Swift

```
// AudioModule.swift
import Foundation

@objc(AudioModule)
class AudioModule: NSObject {

    @objc(play:resolver:rejecter:)
    func play(_ path: String,
              resolver resolve: @escaping RCTPromiseResolveBlock,
              rejecter reject: @escaping RCTPromiseRejectBlock) {
        guard let url = URL(string: path) else {
            reject("bad_path", "Invalid path", nil); return
        }
        // ... start AVAudioPlayer with url ...
        resolve(1200) // duration in ms back to JS
    }

    @objc static func requiresMainQueueSetup() -> Bool { false }
}

// AudioModule.m (Obj-C bridge)
// #import <React/RCTBridgeModule.h>
// @interface RCT_EXTERN_MODULE(AudioModule, NSObject)
// RCT_EXTERN_METHOD(play:(NSString *)path
//   resolver:(RCTPromiseResolveBlock)resolve
//   rejecter:(RCTPromiseRejectBlock)reject)
// @end
```

Q48

NATIVE MODULES & THE NEW ARCHITECTURE

When and how do you wrap a native view, and what is a ViewManager?

THEORY

When you need to render an actual **platform UI control** (a map, a video surface, a camera preview) you wrap it with a **native UI component** rather than a plain native module. On the old architecture this is a **ViewManager** (**RCTViewManager**) that creates the view and exposes props; on Fabric it's a **codegen'd native component** from a spec. You expose props from native to JS and emit events back up.

STRONG ANSWER

If the feature is logic-only I write a native module, but if it has to put a real native view on screen, like a video player surface or a map, I write a native UI component backed by a `ViewManager`. The `ViewManager`'s job is to instantiate the native view and bridge its properties, so a JS prop like volume maps to a setter on the underlying `UIView`, and native events are forwarded back to JS as callbacks. On the old architecture that's `RCTViewManager` with `RCT_EXPORT_VIEW_PROPERTY`; on the New Architecture you describe it in a Codegen spec and get a Fabric component. The mental model is: `ViewManager` is the glue between a JS component and a real native view instance.

CODE · Swift

```
// AudioWaveformViewManager.swift
import UIKit

@objc(AudioWaveformViewManager)
class AudioWaveformViewManager: RCTViewManager {
    override func view() -> UIView! {
        return WaveformView() // your custom UIView subclass
    }
    override static func requiresMainQueueSetup() -> Bool { true }
}

// AudioWaveformViewManager.m
// @interface RCT_EXTERN_MODULE(AudioWaveformViewManager, RCTViewManager)
// RCT_EXPORT_VIEW_PROPERTY(color, NSString)
// RCT_EXPORT_VIEW_PROPERTY(onTap, RCTBubblingEventBlock)
// @end

// JS usage:
// const Waveform = requireNativeComponent('AudioWaveformView');
// <Waveform color="#0af" onTap={e => ...} />
```

ARCHITECTURE / FLOW

```
<Waveform color onTap />      (JS component)
  | props down / events up
  v
ViewManager (creates + configures the native view)
  |
  v
WaveformView : UIView  (the real native control on screen)
```

Q49**NATIVE MODULES & THE NEW ARCHITECTURE****What is autolinking, and how does a native module actually get connected to your app?****THEORY**

Autolinking is the mechanism that automatically discovers native modules in your dependencies and wires them into the iOS and Android builds, so you don't manually edit project files. It reads each package's `react-native.config.js` and metadata, then registers the module during **pod install** (iOS) and **Gradle sync** (Android). Before autolinking, you had to manually link libraries, which was error-prone.

STRONG ANSWER

Autolinking is what saves you from manually editing Xcode projects and Gradle files every time you add a native dependency. When you install a package and run `pod install` on iOS or build on Android, the CLI scans `node_modules`, finds packages that declare native code, and automatically registers them with the build. Each package can describe its native entry points through `react-native.config.js` if it needs custom behavior. For a local module in your own app you point autolinking at the folder, and after a clean rebuild it shows up in `NativeModules`. The thing people forget is that adding a module requires a native rebuild, not just a Metro reload.

CODE · TypeScript

```
// react-native.config.js for a local native module
module.exports = {
  dependencies: {
    'audio-module': {
      root: __dirname + '/native/audio-module',
      platforms: {
        ios: { podspecPath: './native/audio-module/AudioModule.podspec' },
        android: { sourceDir: './native/audio-module/android' },
      },
    },
  },
};

// Then:
// cd ios && pod install      # iOS autolink
// npx react-native run-android # Android autolink via Gradle
// After a native rebuild, NativeModules.AudioModule is available.
```

ARCHITECTURE / FLOW

```
node_modules / local pkgs
|
RN CLI scans + reads react-native.config.js
|
+-----+-----+
v             v
pod install  Gradle sync
(iOS)        (Android)
|
module registered -> available in NativeModules
```

Q50

NATIVE MODULES & THE NEW ARCHITECTURE

How does data and threading work between JS and native, and when do you use callbacks vs promises vs events?

THEORY

Native module methods don't run on the JS thread; they typically run on a **native module queue** (or a queue you specify), so heavy work won't block JS. You return results to JS in three ways: **callbacks** (a one-time function, can fire once), **promises** (resolver/rejecter, ideal for async request/response), and **events** (an emitter, for streaming or multiple values over time). Only **serializable** values cross the boundary.

STRONG ANSWER

By default native methods execute off the JS thread on a background queue, which is good because you don't want a long native operation freezing the UI; if something must touch UIKit you dispatch back to the main thread. For passing results, I pick based on the shape of the interaction: a promise when it's a single async request and response, like `await play(path)`, because it gives clean error handling; a callback for simpler one-shot returns; and an event emitter when native produces a stream of updates over time, like playback progress ticks. For my audio module I used a promise for play and stop, but used events to push periodic position updates to JS. The key constraint is that whatever you pass must be serializable, primitives, arrays, and plain objects.

CODE · TypeScript

```
// Native (Swift) emitting events for streaming updates
@objc(AudioModule)
class AudioModule: RCTEventEmitter {
  override func supportedEvents() -> [String]! { ["onProgress"] }
```


CODE · TypeScript (continued)

```
func tick(_ ms: Int) {
  sendEvent(withName: "onProgress", body: ["positionMs": ms])
}

// JS: promise for request/response, events for streams
import { NativeEventEmitter, NativeModules } from 'react-native';
const { AudioModule } = NativeModules;

const duration = await AudioModule.play('/clip.mp3'); // promise

const emitter = new NativeEventEmitter(AudioModule);
const sub = emitter.addListener('onProgress', e =>
  console.log('pos', e.positionMs)); // events
// sub.remove() when done
```

ARCHITECTURE / FLOW

Native runs OFF the JS thread (module/background queue)

Callback : native --(fires once)--> cb(result)
Promise : await native() -> resolve(value) / reject(err)
Event : native --emit--> emit--> emit--> JS listener (stream)

Only serializable data crosses: numbers, strings, arrays, objects

6

Offline-First, SQLite, Geolocation & Background Tasks

Q51–Q60

SQLite, outbox/sync, background geolocation, backoff and idempotency.

Q51

OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

What does 'offline-first' mean and why does it matter for a mobile app?

THEORY

Offline-first means the app treats the **local device as the source of truth for the UI**, so reads and writes succeed even with no network, and the server sync happens in the background. The network is treated as an **enhancement, not a requirement**, which is critical for mobile where connectivity is unreliable.

STRONG ANSWER

Offline-first means the user can keep working regardless of connectivity, because every action writes to local storage first and syncs to the server later. In my geolocation tracking project this was essential: field users were often in areas with no signal, so the app captured locations into a local SQLite buffer and the UI never blocked on the network. When connectivity returned, a background sync flushed the buffer to the server. This gave us zero data loss and an app that always felt responsive, even on the subway or in a basement.

CODE · TypeScript

```
// Offline-first write: succeed locally, sync later
async function recordLocation(point: LocationPoint) {
  // 1) Always write to the local store first (source of truth)
  await db.execute(
    'INSERT INTO locations (lat, lng, ts, synced) VALUES (?, ?, ?, 0)',
    [point.lat, point.lng, point.ts]
  );
  // 2) UI updates immediately from local data
  emit('location-saved', point);
  // 3) Try to sync, but never block the user on it
  trySyncInBackground().catch(() => {
    /* stays buffered; retried later */
  });
}
```

ARCHITECTURE / FLOW

```
User action
  |
  v
[ Local store ] ---> UI updates instantly (no wait)
  |
  | (background, when online)
  v
[ Server sync ]
```

Q52

OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

How do you choose between AsyncStorage, MMKV, and SQLite for local storage?

THEORY

AsyncStorage is a simple async key-value store, fine for small flags and settings but slow and unstructured. **MMKV** is a synchronous, very fast key-value store backed by memory-mapped files, great for frequently read small data. **SQLite** is a relational database for structured, queryable, large datasets where you need indexes, transactions, and complex queries.

STRONG ANSWER

I pick based on the shape and size of the data. For a few key-value settings like a theme or auth flag I use AsyncStorage or, when I need speed, MMKV which is synchronous and much faster. But for my location tracking project I needed SQLite because I was buffering thousands of GPS points with timestamps and a synced flag, and I had to query 'all unsynced rows ordered by time' and run transactions. That kind of structured, indexed, queryable data is exactly what a relational store is for, while key-value stores would have meant hand-rolling indexes and parsing JSON blobs.

CODE · TypeScript

```
// Rule of thumb in code comments
// AsyncStorage: small, async, simple flags
await AsyncStorage.setItem('onboarded', 'true');

// MMKV: small, synchronous, hot-path reads
import { MMKV } from 'react-native-mmkv';
const storage = new MMKV();
storage.set('lastScreen', 'Map'); // sync, no await
const screen = storage.getString('lastScreen');

// SQLite: structured, large, queryable, transactional
await db.execute(
  'SELECT * FROM locations WHERE synced = 0 ORDER BY ts ASC LIMIT 100'
);
```

ARCHITECTURE / FLOW

```
Small + simple ..... AsyncStorage / MMKV
Small + hot path (sync) . MMKV
Structured + large ..... SQLite
Needs queries/indexes ... SQLite
```

Q53

OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

How do you perform basic CRUD operations with SQLite in React Native?

THEORY

Libraries like **react-native-quick-sqlite** or **op-sqlite** give you a database handle on which you run parameterized SQL. You **always use parameter placeholders (?)** instead of string concatenation to avoid SQL injection and quoting bugs, and wrap multi-row work in **transactions** for speed and atomicity.

STRONG ANSWER

I open the database once, create my tables with an IF NOT EXISTS schema on startup, then run CRUD through parameterized queries. Create is an INSERT, read is a SELECT with a WHERE clause, update sets columns by id, and delete removes by id. In my project the most common operations were inserting a captured location and later updating its synced flag to 1 after a successful upload. I always used placeholders for values and batched inserts in transactions, which kept writes fast even when capturing points every few seconds.

CODE · TypeScript

```
import { open } from '@op-engineering/op-sqlite';
const db = open({ name: 'tracker.db' });

await db.execute(`CREATE TABLE IF NOT EXISTS locations (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  lat REAL, lng REAL, ts INTEGER, synced INTEGER DEFAULT 0
)`);

// CREATE
await db.execute('INSERT INTO locations (lat,lng,ts) VALUES (?, ?, ?)',
  [12.97, 77.59, Date.now()]);
// READ
const { rows } = await db.execute(
  'SELECT * FROM locations WHERE synced = 0');
// UPDATE
await db.execute('UPDATE locations SET synced = 1 WHERE id = ?', [42]);
// DELETE
await db.execute('DELETE FROM locations WHERE id = ?', [42]);
```

Q54**OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS****How would you design an offline write queue (outbox pattern) for pending changes?****THEORY**

The **outbox pattern** stores each pending mutation as a durable row in a local 'outbox' table instead of firing it at the server directly. A separate **flush worker** reads pending rows in order, sends them, and marks or deletes them on success, so writes survive app restarts and network outages.

STRONG ANSWER

I model an outbox table where every offline write becomes a row with a type, a JSON payload, a status, and an attempt count. The app only ever appends to this outbox, and a single flush worker drains it in FIFO order when online. In my geolocation app the buffered location batches were effectively the outbox, and decoupling 'capture' from 'send' this way meant the capture path never failed and never blocked. It also made the system durable, because if the app was killed mid-sync the unsent rows were still on disk and got retried next launch.

CODE · TypeScript

```
// Enqueue any mutation durably
async function enqueue(type: string, payload: object) {
  await db.execute(
    `INSERT INTO outbox (type, payload, status, attempts)
    VALUES (?, ?, 'pending', 0)`,
    [type, JSON.stringify(payload)]
  );
}

// Drain in order
async function flushOutbox() {
  const { rows } = await db.execute(
    `SELECT * FROM outbox WHERE status='pending' ORDER BY id ASC`);
```

CODE · TypeScript (continued)

```

for (const row of rows) {
  try {
    await api.send(row.type, JSON.parse(row.payload));
    await db.execute('DELETE FROM outbox WHERE id=?', [row.id]);
  } catch {
    await db.execute(
      'UPDATE outbox SET attempts = attempts + 1 WHERE id=?', [row.id]);
  }
}
}

```

ARCHITECTURE / FLOW

```

Mutation --> [ outbox table (durable) ]
              | FIFO
              v
          [ flush worker ] --success--> delete row
              |
          failure --> attempts++ (retry later)

```

Q55

OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

What sync strategy and conflict resolution would you use between device and server?

THEORY

When the same record changes in two places you need a rule to decide the winner. **Last-write-wins (LWW)** keeps whichever update has the newest timestamp; it is simple but can silently drop changes. **Versioning** (a version number or vector clock per record) detects true conflicts so you can merge or prompt instead of blindly overwriting.

STRONG ANSWER

I match the strategy to the data. For append-only data like GPS points there is no real conflict, since each reading is a new immutable row, so I just sync them and dedupe. For mutable records I default to last-write-wins using a server-trusted timestamp because it's simple and good enough for most fields. When data is more sensitive I add a version column so the server can reject a stale update and the client can re-fetch and merge. In my tracking project the data was mostly append-only, so I leaned on LWW for the few editable fields and idempotent inserts for the location stream.

CODE · TypeScript

```

// Versioned update: server rejects stale writes
async function pushUpdate(record: Rec) {
  const res = await api.put(`/items/${record.id}`, {
    ...record,
    version: record.version, // version client last saw
  });
  if (res.status === 409) {
    // conflict: server has a newer version
    const fresh = await api.get(`/items/${record.id}`);
    const merged = merge(record, fresh); // or last-write-wins
    return pushUpdate({ ...merged, version: fresh.version });
  }
  return res.data; // accepted; version bumped server-side
}

```

ARCHITECTURE / FLOW

```

Client v3 --update(base=v3)--> Server
      |
      v
server still v3? --> accept, bump to v4
server now v5?  --> 409 conflict
      |
      v
client re-fetch + merge

```

Q56**OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS****How do you capture device location in the background while the app is not in the foreground?****THEORY**

You can't rely on JS timers in the background; you use a **native background-location module** such as `react-native-background-geolocation` or a foreground service. These register an OS-level listener that wakes your handler on location changes or distance/time intervals, and the captured point is written to local storage immediately.

STRONG ANSWER

I use a dedicated background geolocation library that runs in a native service rather than trying to keep JS alive. It emits location updates on movement or interval, and in the handler I do the minimal work of inserting the point into SQLite, which is safe even if the app is backgrounded or killed. The key is to persist on every callback and not depend on in-memory state, plus configure distance filters to balance accuracy against battery. In my project this let us track field users continuously while the app sat in the background, with every point buffered locally for later sync.

CODE · TypeScript

```

import BackgroundGeolocation from 'react-native-background-geolocation';

BackgroundGeolocation.onLocation(async (location) => {
  // Persist immediately; survives backgrounding/kill
  await db.execute(
    'INSERT INTO locations (lat,lng,ts,synced) VALUES (?, ?, ?, 0)',
    [location.coords.latitude, location.coords.longitude, Date.now()]
  );
});

BackgroundGeolocation.ready({
  desiredAccuracy: BackgroundGeolocation.DESIRED_ACCURACY_HIGH,
  distanceFilter: 25,           // meters between updates
  stopOnTerminate: false,      // keep tracking after app close
  startOnBoot: true,           // resume after device reboot
}).then(() => BackgroundGeolocation.start());

```

ARCHITECTURE / FLOW

```

OS location service (native)
  | onLocation callback
  v
[ insert into SQLite (synced=0) ]
  |
  v (later, when online)
[ background sync to server ]

```

Q57

OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

What OS-level constraints affect background tasks on iOS and Android, and how do you work with them?

THEORY

Mobile OSes aggressively limit background execution to save battery. **iOS** gives short background-fetch windows on its own schedule and can suspend or terminate apps, so you can't guarantee periodic timers. **Android** uses Doze mode and background limits, and long-running work usually requires a **foreground service with a persistent notification** or WorkManager-style scheduling.

STRONG ANSWER

The main thing I learned is that you don't control when background work runs; the OS does. On iOS background fetch is opportunistic and brief, so I never assumed an exact interval and instead made sync resumable and idempotent. On Android continuous tracking needs a foreground service with a visible notification, otherwise Doze and background restrictions kill it. So in my project I used a native background-geolocation service for capture, persisted everything to SQLite immediately, and treated any sync opportunity, whether from a fetch window or app foregrounding, as a chance to flush the buffer rather than relying on guaranteed timers.

CODE · TypeScript

```
import BackgroundFetch from 'react-native-background-fetch';

// iOS/Android: opportunistic windows, not exact timers
BackgroundFetch.configure(
  { minimumFetchInterval: 15, stopOnTerminate: false, startOnBoot: true },
  async (taskId) => {
    // Keep work short and idempotent; OS may cut us off
    await flushOutbox();
    BackgroundFetch.finish(taskId); // MUST signal completion
  },
  (taskId) => {
    BackgroundFetch.finish(taskId); // timeout: finish gracefully
  }
);
```

ARCHITECTURE / FLOW

```
iOS: app suspended <----> short fetch window (OS-decided)
Android: Doze / limits --> need foreground service + notif

Strategy: persist locally now, flush on ANY wake opportunity
```

Q58

OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

How do you implement retry with exponential backoff when syncing through network outages?

THEORY

Retrying immediately and forever hammers the server and drains the battery, so you **back off exponentially**: wait roughly $\text{base} * 2^{\text{attempt}}$ between tries, up to a max cap. Adding **jitter** (a random offset) prevents many devices from retrying in lockstep (the thundering-herd problem).

STRONG ANSWER

I retry with an increasing delay so transient failures recover without overwhelming the backend. The delay doubles each attempt up to a ceiling, and I add jitter so devices don't all reconnect at the same instant after an outage. I also cap attempts or fall back to waiting for a connectivity event rather than retrying blindly. In my geolocation app the buffered points stayed safely in SQLite during outages, and the flush worker used backoff so a flaky connection or a brief server hiccup didn't cause a retry storm, while a real reconnect still drained the buffer quickly.

CODE · TypeScript

```
async function flushWithBackoff(maxAttempts = 6) {
  for (let attempt = 0; attempt < maxAttempts; attempt++) {
    try {
      await flushOutbox();
      return; // success
    } catch {
      const base = 1000;           // 1s
      const cap = 60_000;          // 60s ceiling
      const delay = Math.min(cap, base * 2 ** attempt);
      const jitter = Math.random() * delay * 0.3;
      await new Promise((r) => setTimeout(r, delay + jitter));
    }
  }
  // give up for now; data stays buffered for next trigger
}
```

ARCHITECTURE / FLOW

```
attempt: 0    1    2    3    4
delay:  1s   2s   4s   8s  16s ... (cap 60s) + jitter

buffer stays in SQLite the whole time -> no data loss
```

Q59**OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS****How do you detect connectivity changes with NetInfo and react to them?****THEORY**

[@react-native-community/netinfo](#) reports network state and exposes a subscription that fires on changes. You should check **isConnected** and **isInternetReachable**, because a device can be on Wi-Fi yet have no real internet (a captive portal), and trigger a sync on the offline-to-online transition.

STRONG ANSWER

I subscribe to NetInfo and treat the transition from offline to online as the trigger to flush my outbox. I check **isInternetReachable** in addition to **isConnected**, since being attached to a network doesn't always mean the internet works. To avoid spamming the server on flaky connections I debounce the events and only kick off a sync when we go from no connection to a usable one. In my tracking project this is what woke the buffer drain: as soon as a real connection came back, the queued locations started uploading without the user doing anything.

CODE · TypeScript

```
import NetInfo from '@react-native-community/netinfo';

let wasOnline = false;
const unsubscribe = NetInfo.addEventListener((state) => {
  const online = !!state.isConnected && state.isInternetReachable !== false;
  if (online && !wasOnline) {
    // offline -> online transition: drain the buffer
    flushWithBackoff().catch(() => {});
  }
});
```


CODE · TypeScript (continued)

```

    }
    wasOnline = online;
  });

  // On unmount
  // unsubscribe();

```

ARCHITECTURE / FLOW

```

NetInfo events:  offline ... offline ... ONLINE
                  |
                  transition detected (edge)
                  v
                  flush the outbox

```

Q60

OFFLINE-FIRST, SQLITE, GEOLOCATION & BACKGROUND TASKS

How do you ensure data integrity (idempotency and dedup) when flushing the queue to the server?

THEORY

A flush can succeed on the server but fail before the client records it, so the client retries and the server sees the same write twice. **Idempotency** means a repeated request has no extra effect, usually achieved with a **client-generated unique id** that the server dedupes on, so retries are safe and no duplicate rows appear.

STRONG ANSWER

The core risk is double-sending after a response is lost, so I make every item carry a stable client-generated id, like a UUID created at capture time. The server uses that id as an idempotency key and ignores or upserts a record it has already seen, so a retried batch can't create duplicates. On the client I only delete the outbox row after a confirmed success, and I send unique ids rather than relying on auto-increment so the same point is never counted twice. In my geolocation project this guaranteed that flaky retries during outages didn't pollute the dataset with duplicate location points.

CODE · TypeScript

```

import 'react-native-get-random-values';
import { v4 as uuid } from 'uuid';

// Assign a stable id at capture time
async function capture(p: LocationPoint) {
  await db.execute(
    'INSERT INTO locations (client_id, lat, lng, ts) VALUES (?, ?, ?, ?)',
    [uuid(), p.lat, p.lng, p.ts]
  );
}

// Send the client_id so the server can dedupe (idempotent)
async function syncBatch(rows: Row[]) {
  await api.post('/locations/batch', {
    points: rows.map((r) => ({ id: r.client_id, lat: r.lat, lng: r.lng })),
  }); // server upserts by id -> retry-safe
  const ids = rows.map((r) => r.id);
  await db.execute(
    `DELETE FROM locations WHERE id IN (${ids.map(() => '?').join(',')})`,
    ids); // delete only after confirmed success
}

```

ARCHITECTURE / FLOW

```
client_id=abc --send--> server stores abc
| (response lost)
v
client retries client_id=abc --> server sees abc -> ignores
=> exactly one record, no duplicates
```

7

Real-Time: WebSockets, WebRTC & CRDTs

WebSockets, reconnection, Yjs CRDTs, presence, WebRTC and scaling.

Q61–Q70

Q61

REAL-TIME: WEBSOCKETS, WEBRTC & CRDTs

When would you choose HTTP polling, long-polling, SSE, or WebSockets for a real-time feature?

THEORY

Polling repeatedly asks the server on an interval, which is simple but wasteful and laggy. **Long-polling** holds the request open until data is ready, reducing empty responses but still one message per round-trip. **SSE** (Server-Sent Events) is a one-way server-to-client stream over HTTP, great for feeds and notifications. **WebSockets** give a single persistent **full-duplex** connection, which is the right fit for chat, collaboration, and anything bidirectional and high-frequency.

STRONG ANSWER

I pick based on direction and frequency of data. For something like a dashboard that refreshes every minute, plain polling is fine and dead simple. For server-push-only cases like a live notification feed, SSE is lightweight because it rides on normal HTTP and auto-reconnects. But for my collaboration app where both sides constantly exchange edits, presence, and typing events, WebSockets were the only sensible choice since they're full-duplex and avoid per-message HTTP overhead. I reach for WebSockets when the client and server both talk frequently; otherwise I prefer the simpler option.

CODE · TypeScript

```
// Quick decision sketch in code comments
// 1) Polling: simple status checks
setInterval(() => fetch('/api/status').then(r => r.json()), 30000);

// 2) SSE: one-way server push (notifications, feeds)
const es = new EventSource('/api/notifications');
es.onmessage = (e) => console.log('push:', e.data);

// 3) WebSocket: bidirectional, high-frequency (chat, collab)
const ws = new WebSocket('wss://api.example.com/collab');
ws.onmessage = (e) => console.log('msg:', e.data);
ws.onopen = () => ws.send(JSON.stringify({ type: 'join', room: 'doc-1' }));
```

ARCHITECTURE / FLOW

Direction & frequency -> transport

```
one-shot/low freq ..... Polling
delayed pull ..... Long-polling
server -> client ..... SSE
client <-> server ..... WebSocket (chat, collab)
```

```
Polling: C --req--> S C --req--> S (wasteful)
WebSocket: C <=====> S (one persistent full-duplex pipe)
```

Q62

REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

How does a WebSocket connection actually work, from handshake to full-duplex messaging?

THEORY

A WebSocket starts as a normal **HTTP GET** with an ``Upgrade: websocket`` header and a random ``Sec-WebSocket-Key``. The server replies with **101 Switching Protocols** and a hashed ``Sec-WebSocket-Accept``, after which the same TCP socket is reused for the WebSocket protocol. From then on both sides send **frames** independently in either direction, which is what makes it **full-duplex**. The connection stays open until either side sends a close frame.

STRONG ANSWER

It begins life as an HTTP request that asks to upgrade the protocol, so it works through the same ports and proxies as regular web traffic. The server responds with 101 Switching Protocols and from that point the TCP connection is no longer HTTP, it's the WebSocket framing protocol. After the handshake, either side can push messages at any time without waiting for a request, which is the full-duplex part. In my app that meant the server could broadcast another user's edit the instant it arrived, while the client was simultaneously sending its own changes.

CODE · TypeScript

```
// Conceptual handshake (the browser/RN does this for you)
// Client request:
//   GET /collab HTTP/1.1
//   Upgrade: websocket
//   Connection: Upgrade
//   Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
//   Sec-WebSocket-Version: 13
//
// Server response:
//   HTTP/1.1 101 Switching Protocols
//   Upgrade: websocket
//   Connection: Upgrade
//   Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=

const ws = new WebSocket('wss://api.example.com/collab');
ws.onopen = () => console.log('upgraded -> full-duplex open');
ws.onclose = (e) => console.log('closed', e.code, e.reason);
```

ARCHITECTURE / FLOW

WebSocket handshake

CLIENT	SERVER
GET /collab Upgrade:ws ->	
	(validate key)
<- 101 Switching Protocols	
===== full-duplex =====	
-- frame: edit -->	
<-- frame: presence --	
-- close -->	

Q63

REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

How do you set up a WebSocket connection in React Native, including sending messages and cleaning up?

THEORY

React Native ships a global **WebSocket** implementation, so the API matches the browser. The pattern is to open the socket inside an effect, attach ``onopen``, ``onmessage``, ``onerror``, and ``onclose`` handlers, and then **close it in the cleanup** function. Forgetting cleanup is a common leak that leaves duplicate sockets open across re-renders or screen remounts. You also generally want the socket in a ref so re-renders don't recreate it.

STRONG ANSWER

In React Native I treat WebSocket exactly like the browser API since it's built in. I open the connection in a `useEffect`, keep the instance in a ref, and wire up the four event handlers, parsing incoming JSON in `onmessage`. The critical part is the cleanup return that closes the socket when the component unmounts, otherwise navigating between screens stacks up zombie connections. In my collab app I also gated sends behind a `readyState` check so I never tried to write to a socket that wasn't open yet.

CODE · TypeScript

```
import { useEffect, useRef, useCallback } from 'react';

function useCollabSocket(room: string) {
  const wsRef = useRef<WebSocket | null>(null);

  useEffect(() => {
    const ws = new WebSocket(`wss://api.example.com/collab/${room}`);
    wsRef.current = ws;
    ws.onopen = () => ws.send(JSON.stringify({ type: 'join', room }));
    ws.onmessage = (e) => handleMessage(JSON.parse(e.data));
    ws.onerror = (e) => console.warn('ws error', e);
    ws.onclose = () => console.log('ws closed');
    return () => ws.close(1000, 'unmount'); // cleanup
  }, [room]);

  const send = useCallback((msg: object) => {
    const ws = wsRef.current;
    if (ws?.readyState === WebSocket.OPEN) ws.send(JSON.stringify(msg));
  }, []);

  return { send };
}
```

Q64

REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

What reconnection strategy and heartbeat mechanism do you use to keep a WebSocket healthy on mobile?

THEORY

Mobile connections drop constantly when the app backgrounds, switches networks, or hits flaky signal. The standard fix is **exponential backoff with jitter** so reconnect attempts don't hammer the server or sync into a thundering herd. A **heartbeat** (ping/pong) detects dead connections that TCP still thinks are alive, since a silent half-open socket can otherwise hang for minutes. On reconnect you typically re-join rooms and re-sync state.

STRONG ANSWER

I never assume a socket stays open on mobile, so I wrap it in a reconnection loop. On close I schedule a retry with exponential backoff plus a bit of random jitter, capping the delay so it doesn't grow forever. Separately I send a ping on a timer and expect a pong back within a window, and if it's missing I force-close so the backoff loop kicks in and gives me a fresh connection. After reconnecting, my client re-joins the document room and the Yjs provider re-syncs any missed updates, so the user never sees a gap.

CODE · TypeScript

```
function connectWithBackoff(url: string) {
  let attempt = 0;
  let pingTimer: ReturnType<typeof setInterval>;

  const open = () => {
    const ws = new WebSocket(url);
    ws.onopen = () => {
      attempt = 0;
      pingTimer = setInterval(() => {
        if (ws.readyState === WebSocket.OPEN) ws.send('{"type":"ping"}');
      }, 25000);
    };
    ws.onclose = () => {
      clearInterval(pingTimer);
      const base = Math.min(1000 * 2 ** attempt++, 30000); // cap 30s
      const delay = base / 2 + Math.random() * (base / 2); // jitter
      setTimeout(open, delay);
    };
    return ws;
  };
  return open();
}
```

ARCHITECTURE / FLOW

Reconnect backoff with jitter

```
close -> wait ~1s -> try
close -> wait ~2s -> try
close -> wait ~4s -> try
close -> wait ~8s -> try ... capped at 30s
```

Heartbeat:

```
C --ping--> S      (every 25s)
C <--pong-- S      no pong in window -> force close -> backoff
```

Q65**REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS****What is a CRDT and what problem does it solve in a collaborative app?****THEORY**

A **CRDT** (Conflict-free Replicated Data Type) is a data structure designed so that concurrent edits from multiple replicas can always be **merged automatically** without coordination or a central lock. Merges are **commutative**, **associative**, and **idempotent**, meaning order and duplication don't matter, so every replica converges to the same final state. This avoids the classic problem of last-write-wins clobbering someone's changes or needing server-side locks.

STRONG ANSWER

A CRDT is a data type where any two replicas can merge their changes and mathematically guarantee they end up identical, no matter what order updates arrive in. The problem it solves is concurrent editing without a central authority deciding conflicts, which is exactly what you need for offline-first collaboration. In my app two users could edit the same paragraph on different devices, and instead of one overwriting the other, the CRDT interleaved both edits deterministically. I used Yjs for this so I never had to write merge logic by hand, and I never lost a keystroke to a conflict.

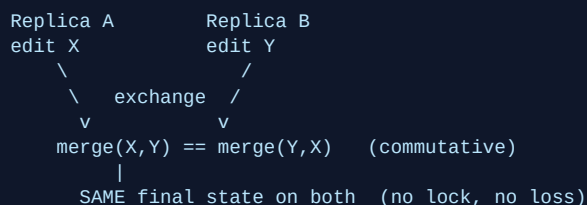
CODE · TypeScript

```
// Why last-write-wins fails vs why CRDTs converge
// LW: two offline edits to same field
//   A sets title = 'Roadmap'   @ t1
//   B sets title = 'Q3 Plan'   @ t2  -> A's edit lost

// CRDT text: each char has a unique id + position
// concurrent inserts are ORDERED deterministically, not dropped
import * as Y from 'yjs';
const a = new Y.Doc();
const b = new Y.Doc();
a.getText('t').insert(0, 'Hello');
b.getText('t').insert(0, 'Hi');
// exchange updates both ways:
Y.applyUpdate(b, Y.encodeStateAsUpdate(a));
Y.applyUpdate(a, Y.encodeStateAsUpdate(b));
console.log(a.getText('t').toString() === b.getText('t').toString());
// -> true (both converge to the same merged string)
```

ARCHITECTURE / FLOW

CRDT convergence

**Q66**

REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

Can you explain the basics of Yjs: shared types, updates, and providers?**THEORY**

Yjs centers on a **Y.Doc**, a container holding **shared types** like `Y.Text`, `Y.Array`, and `Y.Map` that behave like normal data structures but are CRDT-backed. Changes produce compact binary **updates** that can be applied to any other doc to sync it, and updates are encoded as deltas of the document state. **Providers** are pluggable transports (such as y-websocket or y-webrtc) that move those updates between peers and often handle persistence and awareness.

STRONG ANSWER

The core object is a `Y.Doc`, and inside it you grab shared types like `Y.Text` or `Y.Map` that you mutate like ordinary JS, but every mutation is tracked as a CRDT operation. Those mutations get serialized into small binary updates, and applying an update to another doc merges it in, which is the whole sync mechanism. Providers are the layer that actually ships updates around. In my app I used the `WebSocket` provider so the server relayed updates to everyone in a room, and the same updates were persisted so a fresh client could load the document state.

CODE · TypeScript

```
import * as Y from 'yjs';
import { WebSocketProvider } from 'y-websocket';

const doc = new Y.Doc();

// shared types behave like normal structures, but are CRDTs
const text = doc.getText('content');
const meta = doc.getMap('meta');

// provider = transport that syncs updates to all peers in the room
const provider = new WebSocketProvider(
  'wss://api.example.com', 'doc-room-42', doc
);

// observe remote + local changes
text.observe(() => render(text.toString()));

// local edits -> binary update -> broadcast by provider automatically
text.insert(0, 'Sprint goals: ');
meta.set('updatedAt', Date.now());

provider.on('status', (e: { status: string }) => console.log(e.status));
```

ARCHITECTURE / FLOW

```
Yjs pieces

Y.Doc
|-- Y.Text  (content)  <- shared CRDT types
|-- Y.Map   (meta)
|-- Y.Array (items)
|
| mutate -> binary UPDATE (delta)
|
Provider (y-websocket) -- relays update -> all peers in room
```

Q67

REAL-TIME: WEBSOCKETS, WEBRTC & CRDTs

How did you implement live presence and typing indicators?

THEORY

Presence is **ephemeral state** that doesn't belong in the persisted document, things like cursor position, name, color, and an is-typing flag. Yjs handles this through its **awareness** protocol, a separate channel that broadcasts each client's transient state and automatically clears it when they disconnect. Typing indicators are usually a boolean in awareness, **debounced** so they flip off shortly after the user stops typing.

STRONG ANSWER

I kept presence completely out of the CRDT document because it's throwaway data, and used the Yjs awareness API instead. Each client sets its local awareness state with the user's name, color, and cursor, and listens for changes to render everyone's presence. For typing indicators I set an isTyping flag on keypress and used a debounce timer to clear it after about a second of inactivity. The nice part is awareness automatically drops a user's state when their socket closes, so stale typing bubbles and ghost cursors disappear on their own without me writing cleanup logic.

CODE · TypeScript

```
import { WebSocketProvider } from 'y-websocket';

const awareness = provider.awareness; // ephemeral, not in the doc
```


CODE · TypeScript (continued)

```

awareness.setLocalStateField('user', { name: 'Rinshad', color: '#3b82f6' });

let typingTimer: ReturnType<typeof setTimeout>;
function onKeyPress() {
  awareness.setLocalStateField('typing', true);
  clearTimeout(typingTimer);
  typingTimer = setTimeout(
    () => awareness.setLocalStateField('typing', false),
    1000 // debounce: clear shortly after typing stops
  );
}

// render everyone's presence; disconnected peers auto-removed
awareness.on('change', () => {
  const peers = Array.from(awareness.getStates().values());
  renderPresence(peers); // names, cursors, typing bubbles
});

```

ARCHITECTURE / FLOW

Awareness (ephemeral) vs Doc (persisted)

```

Y.Doc      ---> persisted, merged forever (text, data)
Awareness  ---> transient: cursor, name, typing flag
              auto-cleared on disconnect

keypress -> typing=true --(debounce 1s)--> typing=false

```

Q68

REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

How do conflict-free offline edits work, and how do they merge when the device comes back online?

THEORY

Because Yjs is a CRDT, an **offline replica** can keep accepting edits with no server contact, storing them locally. Each edit carries enough identity (client id + sequence) that it can be merged later regardless of what others did meanwhile. On reconnect, the client exchanges a compact **state vector** so each side sends only the updates the other is missing, and the merge is **deterministic** so all replicas converge. No edits are dropped and no manual conflict resolution is needed.

STRONG ANSWER

Offline, the Yjs doc just keeps mutating in memory and I persisted it to local storage so edits survived an app restart. When the device reconnected, the provider performed a sync where both sides exchange state vectors and then only the missing updates, so it's efficient even after a long offline stretch. The CRDT guarantees that my queued offline edits and everyone else's changes merge into the same result on every device. In practice a user could go through a tunnel, keep editing, and come back to find their work and their teammates' work both intact and interleaved correctly.

CODE · TypeScript

```

import * as Y from 'yjs';

// while offline, edits accumulate locally and are persisted
function queueOfflineEdit(doc: Y.Doc) {
  doc.getText('content').insert(0, 'offline note ');
  localStorage.setItem('doc', JSON.stringify(
    Array.from(Y.encodeStateAsUpdate(doc))
  ));
}

```

CODE · TypeScript (continued)

```
// on reconnect: send only what the other side lacks (state vector diff)
function sync(local: Y.Doc, remoteStateVector: Uint8Array) {
  // give peer exactly the updates they're missing
  const diff = Y.encodeStateAsUpdate(local, remoteStateVector);
  ws.send(diff);
}

function applyRemote(local: Y.Doc, update: Uint8Array) {
  Y.applyUpdate(local, update); // deterministic merge, nothing lost
}
```

ARCHITECTURE / FLOW

Offline edit + merge on reconnect

```
Device (offline)      Server / peers
edit A                edit B, edit C
edit A2      [no network]
|  reconnect: exchange state vectors
|--- missing: A, A2 ----->
|<-- missing: B, C -----
v
merge(A,A2,B,C) deterministic -> identical state everywhere
```

Q69

REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

Walk me through WebRTC basics, peer connection, signaling, and STUN/TURN, for sending voice notes.

THEORY

WebRTC sets up a direct peer-to-peer media/data channel between two clients. The peers first exchange **SDP offer/answer** describing codecs and media, plus **ICE candidates** (possible network paths), through a separate **signaling** channel you provide, often a WebSocket. **STUN** servers help a peer discover its public address behind NAT, and when direct connection fails, a **TURN** server relays the media as a fallback. The signaling server only brokers the setup; the actual audio flows peer-to-peer once connected.

STRONG ANSWER

WebRTC itself doesn't define how peers find each other, so I used my existing WebSocket as the signaling channel to pass the SDP offer and answer plus ICE candidates between the two clients. Each side creates an `RTCPeerConnection`, the caller makes an offer, the callee answers, and they trade ICE candidates until they agree on a path. STUN lets each peer learn its public-facing address through NAT, which covers most cases, and TURN is the relay fallback for restrictive networks where a direct path can't be established. For voice notes specifically I could attach a microphone track to the connection, and importantly TURN matters in production because a meaningful share of mobile users sit behind symmetric NATs.

CODE · TypeScript

```
const pc = new RTCPeerConnection({
  iceServers: [
    { urls: 'stun:stun.l.google.com:19302' },
    { urls: 'turn:turn.example.com', username: 'u', credential: 'p' },
  ],
});

// attach mic for the voice note
const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
stream.getTracks().forEach((t) => pc.addTrack(t, stream));
```

CODE · TypeScript (continued)

```
// signaling over the existing WebSocket
pc.onicecandidate = (e) => e.candidate && signal({ ice: e.candidate });

// caller side
const offer = await pc.createOffer();
await pc.setLocalDescription(offer);
signal({ sdp: offer }); // send via WS; peer replies with an answer
```

ARCHITECTURE / FLOW

WebRTC setup

Peer A	Signaling (WS)	Peer B
-- offer (SDP) ----->	-- offer ----->	
<----- answer (SDP) --	<-- answer -----	
-- ICE candidates ---->	<-- ICE ----->	

Path discovery:

STUN -> learn public IP (direct P2P if possible)

TURN -> relay audio when direct path is blocked

Once connected: AUDIO flows A <==P2P==> B (not via server)

Q70

REAL-TIME: WEBSOCKETS, WEBRTC & CRDTS

How do you scale a real-time backend across many servers, and why do you need Redis or a message broker?

THEORY

A single WebSocket server only knows about the clients **connected to it**, so once you run multiple instances behind a load balancer, a broadcast on one server won't reach users on another. The fix is a **pub/sub** layer (commonly **Redis Pub/Sub** or a broker like NATS/Kafka) where each server publishes room events and all servers subscribe, fanning messages out to their local sockets. **Rooms** scope messages so only relevant clients get them, and sticky sessions or a shared store keep connection state coherent. This decouples message routing from any single node.

STRONG ANSWER

The moment you scale past one box, you hit the fan-out problem: a user on server A and their teammate on server B are in the same document room but their sockets live on different machines. So each server can't just broadcast locally, it has to publish the event to a shared pub/sub layer that every server subscribes to, and each server then pushes to its own connected clients in that room. I'd use Redis Pub/Sub for this because it's simple and fast, or a durable broker like Kafka if I needed replay or guaranteed delivery. Rooms keep the fan-out targeted so you're not blasting every event to every user, and that's how the collab app could in principle support many documents across a cluster.

CODE · TypeScript

```
import { createClient } from 'redis';

const pub = createClient();
const sub = pub.duplicate();
await pub.connect();
await sub.connect();

// each server subscribes to room channels
await sub.pSubscribe('room:', (message, channel) => {
  const room = channel.split(':')[1];
  // deliver to sockets connected to THIS server in that room
  localSockets.get(room)?.forEach((ws) => ws.send(message));
});
```

CODE · TypeScript (continued)

```
// when a client sends an edit, publish so ALL servers fan it out
function onClientMessage(room: string, payload: string) {
  pub.publish(`room:${room}`, payload);
}
```

ARCHITECTURE / FLOW

Scaling fan-out via pub/sub

```
Client A --- WS ---> Server 1      Server 2 <--- WS --- Client B
      |                |
      v                v
    publish room:42    subscribe room:42
      \                /
       > Redis Pub/Sub <
(Server 1 publishes; Server 2 receives and pushes to Client B)
Rooms scope delivery; broker decouples routing from any node
```

8

AI / LLM Features on Mobile

Streaming chat, on-device Whisper, tool calling, RAG, embeddings & VAD.

Q71–Q80

Q71

AI / LLM FEATURES ON MOBILE

How does LLM streaming work, and why would you stream a response instead of waiting for the full reply?

THEORY

LLMs generate text **one token at a time** (a token is roughly a word-piece), so the server can emit each token the moment it is produced rather than buffering the whole answer. Most APIs deliver this over **Server-Sent Events (SSE)**, a long-lived HTTP response where the server pushes `data:` lines until a final `[DONE]` marker. Streaming cuts perceived latency dramatically because the user sees the first words in a few hundred milliseconds instead of waiting several seconds for a full paragraph.

STRONG ANSWER

In my voice assistant the model generates tokens incrementally, so I consume them over an SSE stream instead of awaiting the complete response. The big win is perceived latency: time-to-first-token is well under a second, so the chat bubble starts filling in immediately while the model is still working. That also lets me kick off text-to-speech on partial sentences, which makes the whole voice loop feel responsive. Both Gemini and Groq expose streaming endpoints, and Groq in particular streams extremely fast, so for a conversational UI streaming is basically mandatory rather than optional.

CODE · TypeScript

```
// Anatomy of an SSE token stream from an LLM endpoint
// Each event line looks like:
//
// data: {"choices":[{"delta":{"content":"Hel"}}]}
// data: {"choices":[{"delta":{"content":"lo"}}]}
// data: [DONE]

function parseSSEChunk(raw: string): string[] {
  const tokens: string[] = [];
  for (const line of raw.split("\n")) {
    if (!line.startsWith("data:")) continue;
    const payload = line.slice(5).trim();
    if (payload === "[DONE]") break;
    try {
      const json = JSON.parse(payload);
      const delta = json.choices?.[0]?.delta?.content;
      if (delta) tokens.push(delta);
    } catch {
      // partial JSON across chunk boundary; buffer & retry upstream
    }
  }
  return tokens;
}
```

ARCHITECTURE / FLOW

```
Non-streaming: request -----[ wait 4s ]-----> full reply

Streaming:      request -> tok -> tok -> tok -> ... -> [DONE]
                  ^first token ~300ms (UI updates live)
```

Q72

AI / LLM FEATURES ON MOBILE

How do you implement streaming chat in React Native, consuming the stream and rendering tokens as they arrive?

THEORY

React Native's default `fetch` does not expose a streaming body reader on all platforms, so the common approach is a library like **react-native-sse** or `fetch` with **XHR-based streaming** to read `data:` events as they come. As each token arrives you **append it to state** and let React re-render the growing message. To avoid jank you batch updates (or use a ref + throttled flush) so you aren't calling `setState` on every single token.

STRONG ANSWER

I use react-native-sse to open the SSE connection and listen for message events. As each delta comes in I append it to the in-progress assistant message in state, so the bubble grows token by token like ChatGPT. To keep it smooth I don't call setState on literally every token; I accumulate into a ref and flush on an interval or animation frame, which avoids re-rendering the whole list dozens of times a second. I also keep the EventSource handle so I can cancel mid-stream if the user sends a new message or taps stop, and I handle the error and close events to gracefully finalize the bubble.

CODE · TypeScript

```
import EventSource from "react-native-sse";

function streamChat(prompt: string, onToken: (t: string) => void) {
  const es = new EventSource("https://api.groq.com/openai/v1/chat/completions", {
    method: "POST",
    headers: { Authorization: `Bearer ${KEY}`, "Content-Type": "application/json" },
    body: JSON.stringify({
      model: "llama-3.1-8b-instant",
      stream: true,
      messages: [{ role: "user", content: prompt }],
    }),
  });

  es.addEventListener("message", (e) => {
    if (e.data === "[DONE]") return es.close();
    const delta = JSON.parse(e.data!).choices?.[0]?.delta?.content;
    if (delta) onToken(delta); // append to buffered ref, flush on rAF
  });
  es.addEventListener("error", () => es.close());
  return () => es.close(); // call to cancel an in-flight stream
}
```

ARCHITECTURE / FLOW

```
SSE event -> onToken() -> buffer ref --(throttled flush)-->
setState -> FlatList re-render -> bubble grows live
```

Q73

AI / LLM FEATURES ON MOBILE

Why run speech-to-text on-device with Whisper (whisper.cpp) instead of calling a cloud transcription API?

THEORY

Whisper is OpenAI's open speech recognition model, and **whisper.cpp** is a C/C++ port that runs quantized model weights efficiently on a phone's CPU/GPU. Running it on-device means audio never leaves the phone, which is a **privacy** win and removes network round-trips for **lower latency**. It also keeps **cost** at zero (no per-minute API billing) and works offline, at the price of bundling a model file and using more battery/CPU.

STRONG ANSWER

I integrated whisper.cpp via a React Native binding so transcription happens entirely on the device using a small quantized model like ggml-tiny or base. The main reasons were privacy and latency: the user's voice never goes to a server, and I avoid a network hop before the LLM even sees the text. It's also free, which matters since I'm on free-tier APIs and didn't want a metered transcription bill. The trade-off is a larger app bundle for the model and heavier CPU use, so I picked a quantized model size that balances accuracy against speed on mid-range phones.

CODE · TypeScript

```
import { initWhisper } from "whisper.rn";

let ctx: Awaited<ReturnType<typeof initWhisper>>;

export async function loadModel() {
  // quantized ggml model bundled with the app
  ctx = await initWhisper({ filePath: require("../assets/ggml-base.q5.bin") });
}

export async function transcribe(wavPath: string): Promise<string> {
  const { promise } = ctx.transcribe(wavPath, {
    language: "en",
    maxThreads: 4, // tune for the device's core count
    translate: false,
  });
  const { result } = await promise; // runs fully offline, no network
  return result.trim();
}
```

ARCHITECTURE / FLOW

```
Cloud STT: mic -> upload audio -> server -> text (privacy + RTT cost)

On-device: mic -> whisper.cpp (phone CPU) -> text (private, offline, $0)
```

Q74

AI / LLM FEATURES ON MOBILE

How do you handle text-to-speech playback so the assistant can speak its replies?

THEORY

Text-to-speech (TTS) converts the model's text answer into audio. On mobile you can use the **OS-native TTS engine** (via expo-speech or react-native-tts) for zero cost and offline support, or a **cloud TTS** for more natural voices at the price of latency and billing. A key UX trick with streaming LLMs is to **speak sentence-by-sentence** as tokens arrive rather than waiting for the full reply, so the assistant starts talking almost immediately.

STRONG ANSWER

For the voice assistant I use the native TTS engine through expo-speech, which is free, offline, and low-latency. The important part is that I don't wait for the whole LLM response: as the stream produces text I split it on sentence boundaries and queue each finished sentence to TTS, so the assistant begins speaking while the model is still generating. I also manage state carefully, stopping any in-progress speech when the user starts talking again so the assistant doesn't talk over them. If I wanted more natural voices I'd swap in a cloud TTS behind the same interface, but native was the right call for a free-tier, responsive app.

CODE · TypeScript

```
import * as Speech from "expo-speech";

let buffer = "";

// Feed streaming LLM tokens; speak each completed sentence immediately.
export function feedTokenToTTS(token: string) {
  buffer += token;
  const match = buffer.match(/^(.*?[.!?])\s+(.*)$/s);
  if (match) {
    const [, sentence, rest] = match;
    Speech.speak(sentence, { language: "en", rate: 1.0 });
    buffer = rest; // keep remainder for the next sentence
  }
}

export function flushTTS() {
  if (buffer.trim()) Speech.speak(buffer.trim());
  buffer = "";
}

export function stopTTS() {
  Speech.stop(); // call when user starts speaking (barge-in)
}
```

Q75**AI / LLM FEATURES ON MOBILE****What is tool calling (function calling), and how do you wire it up with an LLM?****THEORY**

Tool calling lets the model **request a function** instead of answering directly: you describe your tools as a **JSON schema** (name, description, parameters), and when the model decides one is needed it returns a structured ``tool_call`` with arguments. Your app **executes the real function**, feeds the result back into the conversation, and the model uses it to produce the final reply. This is how an LLM safely reaches out to the live world (weather, search, your database) without hallucinating the data.

STRONG ANSWER

In my assistant I expose a few tools, like fetching reminders or doing a search, each defined with a JSON schema for its name and parameters. When the model decides it needs one, the stream returns a `tool_call` with JSON arguments instead of text; I parse those, run the actual function, and append the result back as a tool message. Then I call the model again so it can answer using the real data. The key things I watch are validating the arguments before executing (the model can produce malformed or unexpected values) and handling the multi-turn loop, since a single user question can trigger several tool calls before a final answer.

CODE · TypeScript

```
const tools = [{
  type: "function",
  function: {
    name: "get_reminders",
    description: "Fetch the user's reminders for a given day",
    parameters: {
      type: "object",
      properties: { date: { type: "string", description: "ISO date" } },
      required: ["date"],
    },
  },
},
];

async function handleToolCalls(call: { name: string; arguments: string }) {
  const args = JSON.parse(call.arguments); // validate before trusting!
  if (call.name === "get_reminders") {
    const data = await db.getReminders(args.date);
    return { role: "tool", name: call.name, content: JSON.stringify(data) };
  }
  throw new Error(`Unknown tool: ${call.name}`);
}
```

ARCHITECTURE / FLOW

```
user -> LLM --(needs data)--> tool_call{name,args}
      ^                      |
      |                      | app runs fn
      |                      v
final answer <-- LLM <-- tool result (fed back)
```

Q76

AI / LLM FEATURES ON MOBILE

What is RAG (Retrieval-Augmented Generation), and why do you need it?

THEORY

RAG means retrieving relevant chunks of your own knowledge base and **injecting them into the prompt** so the model answers from real, current data instead of only its training. You need it because LLMs have a fixed knowledge cutoff, don't know your private content, and can **hallucinate** when asked about things outside their training. By grounding the answer in retrieved context you get accurate, source-backed responses without retraining the model.

STRONG ANSWER

I built a RAG pipeline so the assistant can answer questions about the user's own documents rather than guessing. At ingest time I chunk the documents, embed each chunk with Gemini embeddings, and store the vectors in PostgreSQL with pgvector. At query time I embed the user's question, run a cosine similarity search to pull the most relevant chunks, and stuff those into the system prompt as context before calling the LLM. This fixes the two big problems with raw LLMs: it grounds answers in real data so they don't hallucinate, and it lets me serve private and up-to-date content the base model never saw, all without any fine-tuning.

CODE · TypeScript

```
// RAG query flow: embed -> search -> assemble -> generate
async function answerWithRAG(question: string) {
  const qVec = await embed(question); // Gemini embedding
  const chunks = await searchSimilar(qVec, 5); // pgvector cosine

  const context = chunks.map((c, i) => `[${i + 1}] ${c.text}`).join("\n\n");
  const messages = [
    { role: "system", content: "Answer ONLY from the context below. If unknown, say so.\n\n" + context },
    { role: "user", content: question },
  ],
```

CODE · TypeScript (continued)

```
];
return streamChat(messages); // grounded, source-backed answer
}
```

ARCHITECTURE / FLOW

```
ingest: docs -> chunk -> embed -> store vectors (pgvector)

query:  question -> embed -> vector search -> top-k chunks
        -> inject into prompt -> LLM -> grounded answer
```

Q77

AI / LLM FEATURES ON MOBILE

How do embeddings and vector similarity search work, and what do cosine distance, pgvector, and an HNSW index give you?

THEORY

An **embedding** maps text to a high-dimensional vector so that semantically similar text lands close together in that space. **Cosine similarity** measures the angle between two vectors, so it ranks by meaning regardless of length, and finding the closest stored vectors is a **nearest-neighbor search**. **pgvector** adds a `vector` column type to PostgreSQL, and an **HNSW index** (a graph-based approximate-nearest-neighbor structure) makes that search fast at scale instead of scanning every row.

STRONG ANSWER

In my pipeline each chunk and each query becomes a Gemini embedding, and similarity is just cosine distance between those vectors. I store them in a pgvector column and query with the `<=>` cosine operator to get the top-k most relevant chunks. Without an index, that's a full sequential scan over every vector, which gets slow as the table grows, so I added an HNSW index, which is an approximate nearest-neighbor graph that trades a tiny bit of recall for a large speedup. I tune the index parameters and the `ef_search` setting to balance recall against latency, which matters because retrieval is on the critical path before the LLM even runs.

CODE · SQL

```
-- pgvector table with an HNSW index for fast cosine search
CREATE TABLE chunks (
  id      bigserial PRIMARY KEY,
  text    text,
  embedding vector(768)      -- Gemini embedding dimension
);

CREATE INDEX ON chunks
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- top-5 most similar chunks to the query vector ($1)
SET hnsw.ef_search = 40;      -- higher = better recall, slower
SELECT id, text, 1 - (embedding <=> $1) AS similarity
FROM chunks
ORDER BY embedding <=> $1      -- <=> is cosine distance
LIMIT 5;
```

ARCHITECTURE / FLOW

```
vector space (meaning)
query *                cat o
  \ cosine angle small /
  \---- nearest neighbors ----> kitten o
```

ARCHITECTURE / FLOW (continued)

```

      \                puppy o
HNSW graph hops between nodes -> finds top-k fast

```

Q78

AI / LLM FEATURES ON MOBILE

What are the basics of prompt engineering you rely on — system prompt, context injection, and token limits?

THEORY

The **system prompt** sets the model's role, rules, and tone for the whole conversation and is the strongest lever on behavior. **Context injection** means inserting retrieved data or conversation history into the prompt so the model has the facts it needs at generation time. Every model has a **context window** measured in tokens, so you must budget how much system text, history, and retrieved context you include, or older content gets truncated and the model loses track.

STRONG ANSWER

My system prompt defines the assistant's persona and hard rules, like answering only from provided context and admitting when it doesn't know, which cuts down hallucination a lot. For context injection I assemble the RAG chunks and a trimmed slice of recent conversation into the prompt right before the user's question. I'm always mindful of token limits: the context window is finite, so I cap how many chunks I inject and summarize or drop old turns rather than blowing past the limit and getting truncated. Keeping the system prompt tight also matters for cost and latency on free-tier APIs, since every token I send is a token I pay for in time.

CODE · TypeScript

```

const MAX_CONTEXT_TOKENS = 6000;
const est = (s: string) => Math.ceil(s.length / 4); // rough token estimate

function buildPrompt(system: string, chunks: string[], history: Msg[], q: string) {
  let budget = MAX_CONTEXT_TOKENS - est(system) - est(q);
  const ctx: string[] = [];
  for (const c of chunks) { // inject highest-ranked chunks first
    if (budget - est(c) < 0) break;
    ctx.push(c); budget -= est(c);
  }
  const recent: Msg[] = [];
  for (let i = history.length - 1; i >= 0; i--) { // newest history that fits
    if (budget - est(history[i].content) < 0) break;
    recent.unshift(history[i]); budget -= est(history[i].content);
  }
  return [{ role: "system", content: `${system}\n\nContext:\n${ctx.join("\n")}` },
    ...recent, { role: "user", content: q }];
}

```

ARCHITECTURE / FLOW

```

[ context window budget (tokens) ]
| system prompt | retrieved context | history | user q |
^fixed rules   ^top-k chunks      ^trim old ^always kept
if it overflows -> oldest history dropped first

```

Q79

AI / LLM FEATURES ON MOBILE

How do you manage latency and cost on free-tier APIs like Gemini and Groq, including caching?

THEORY

Free-tier APIs impose **rate limits and quotas** (requests/tokens per minute), so you must avoid redundant calls and degrade gracefully when throttled. **Caching** identical or near-identical requests, **streaming** to lower perceived latency, and **routing** simple work to the fastest/cheapest model are the main levers. Trimming prompt size and embedding-caching also directly reduce both token cost and round-trip time.

STRONG ANSWER

Because I'm on Gemini and Groq free tiers, I treat every call as a scarce resource. I cache embeddings so identical chunks or repeated queries never get re-embedded, and I cache full responses for repeated questions keyed on the prompt. For latency I lean on streaming and route to Groq for fast chat since it's extremely quick, while using Gemini for embeddings. I also keep prompts lean to stay under token limits, batch where possible, and handle 429 rate-limit errors with exponential backoff plus a fallback, so a quota spike degrades gracefully instead of breaking the conversation.

CODE · TypeScript

```
const cache = new Map<string, string>(); // prompt -> response

async function cachedComplete(prompt: string): Promise<string> {
  const key = hash(prompt);
  if (cache.has(key)) return cache.get(key)!; // skip the API entirely

  for (let attempt = 0; attempt < 4; attempt++) {
    const res = await fetch(GROQ_URL, { method: "POST", body: bodyFor(prompt) });
    if (res.status === 429) { // rate limited
      await sleep(2 ** attempt * 500); // exponential backoff
      continue;
    }
    const text = (await res.json()).choices[0].message.content;
    cache.set(key, text);
    return text;
  }
  throw new Error("rate limited; fall back to cached/older model");
}
```

ARCHITECTURE / FLOW

```
request -> [cache hit?] --yes--> instant, $0
      |no
      v
  API call -> 429? -> backoff & retry -> cache result
```

Q80

AI / LLM FEATURES ON MOBILE

What is Voice Activity Detection (VAD), and how does the end-to-end voice pipeline fit together (mic to VAD to Whisper to LLM to TTS)?

THEORY

Voice Activity Detection distinguishes speech from silence/noise in the mic stream, so you know **when the user starts and stops talking** without a push-to-talk button. It triggers transcription only on real speech, which saves CPU and avoids feeding silence to Whisper. In the full loop, VAD gates the audio, Whisper turns speech into text, the LLM generates a streamed answer, and TTS speaks it back, ideally with **barge-in** so the user can interrupt.

STRONG ANSWER

VAD is the piece that makes my assistant hands-free: instead of a button, it listens to the mic and detects when speech starts and, more importantly, when the user has stopped, using an end-of-speech timeout. When VAD signals end-of-utterance, I capture that audio segment and hand it to on-device Whisper for transcription. That text goes to the LLM, which streams an answer, and I pipe completed sentences into TTS so the assistant starts speaking quickly. I also wire barge-in: if VAD detects the user talking again, I stop TTS and any in-flight LLM stream, which keeps the conversation feeling natural rather than turn-locked.

CODE · TypeScript

```
// Orchestrating the voice loop, gated by VAD events
vad.on("speechStart", () => {
  stopTTS(); // barge-in: user interrupts the assistant
  cancelStream?.(); // abort any in-flight LLM stream
});

vad.on("speechEnd", async (segment: AudioSegment) => {
  const text = await transcribe(segment.wavPath); // on-device Whisper
  if (!text) return;

  cancelStream = streamChat(buildPrompt(text), (token) => {
    appendToBubble(token); // render live
    feedTokenToTTS(token); // speak sentence-by-sentence
  });
});
```

ARCHITECTURE / FLOW

```
mic stream
|
[ VAD ] --silence--> ignore
| speech end
v
[ Whisper STT ] -> text -> [ LLM (stream) ] -> tokens
|                                     |
| appendToBubble                     [ TTS ] -> speaker
user speaks again -> VAD speechStart -> barge-in: stop TTS+stream
```

9

Backend & Data

REST, Express, JWT rotation, SQL vs NoSQL, indexing, Redis & idempotency.

Q81–Q90

Q81

BACKEND & DATA

How do you design a clean REST API in terms of resources, HTTP verbs, and status codes?

THEORY

REST models the system as **resources** addressed by nouns in the URL, not verbs. The **HTTP method conveys the action**: GET reads, POST creates, PUT/PATCH update, DELETE removes. **Status codes communicate outcome** so clients can react programmatically without parsing the body: 2xx success, 4xx client error, 5xx server error.

STRONG ANSWER

I design URLs around resources, so I'd use /users and /users/:id/orders rather than verbs like /getUserOrders. The method carries the intent: POST to create returns 201 with the new resource, GET returns 200, a missing record returns 404, bad input returns 400, and auth failures return 401 or 403. I keep responses consistent with a predictable shape and use plural nouns, nested routes for relationships, and query params for filtering and pagination. On my Express backend this consistency made the mobile client code much simpler because the app could rely on status codes instead of reading error strings.

CODE · TypeScript

```
// Resource-oriented Express routes
import { Router } from 'express';
const router = Router();

router.get('/users/:id/orders', async (req, res) => {
  const orders = await orderService.listByUser(req.params.id);
  return res.status(200).json({ data: orders });
});

router.post('/users/:id/orders', async (req, res) => {
  const order = await orderService.create(req.params.id, req.body);
  // 201 + Location header for the new resource
  res.location(`/orders/${order.id}`);
  return res.status(201).json({ data: order });
});

router.delete('/orders/:orderId', async (req, res) => {
  const removed = await orderService.remove(req.params.orderId);
  if (!removed) return res.status(404).json({ error: 'Not found' });
  return res.status(204).send(); // no content
});

export default router;
```

ARCHITECTURE / FLOW

```
Verb + Resource -> Status
GET    /users/:id/orders    -> 200 list
POST   /users/:id/orders    -> 201 created (+Location)
PATCH /orders/:id          -> 200 updated
DELETE /orders/:id          -> 204 / 404 if missing
(bad body 400 | no token 401 | forbidden 403)
```

Q82

BACKEND & DATA

What is Express middleware and how does the request flow through it?

THEORY

Express processes each request through an ordered **chain of middleware functions**, each receiving (req, res, next). A middleware can run logic, mutate req/res, end the response, or call **next()** to **pass control** to the next handler.

Order matters: middleware registered earlier runs first, which is why auth and body parsing sit before route handlers and error handlers sit last.

STRONG ANSWER

Middleware is just a function with req, res, and next that sits in the pipeline before or around your route handler. I use it for cross-cutting concerns like JSON parsing, request logging, authentication, and rate limiting, so the controller stays focused on business logic. The key rule is that you must either send a response or call next, otherwise the request hangs. Error-handling middleware is special because it takes four arguments, err first, and I register it last so any thrown error funnels into one consistent error response. This layered approach kept my route handlers thin and made the auth and validation logic reusable across every endpoint.

CODE · TypeScript

```
import express, { Request, Response, NextFunction } from 'express';
const app = express();

app.use(express.json()); // body parser middleware

// custom logging middleware
app.use((req: Request, _res: Response, next: NextFunction) => {
  console.log(`${req.method} ${req.path}`);
  next(); // pass control onward
});

function requireAuth(req: Request, res: Response, next: NextFunction) {
  if (!req.headers.authorization) return res.status(401).json({ error: 'No token' });
  next();
}

app.get('/profile', requireAuth, (req, res) => {
  res.json({ data: { id: 1 } });
});

// error-handling middleware (4 args) registered LAST
app.use((err: Error, _req: Request, res: Response, _next: NextFunction) => {
  res.status(500).json({ error: err.message });
});
```

ARCHITECTURE / FLOW

```
Request
  |
  v
[express.json] -> [logger] -> [requireAuth] -> [controller]
  |                                     |
  | (any throw / next(err))           |
  +-----> [error handler] <-- Response
```

Q83

BACKEND & DATA

What is the difference between access tokens and refresh tokens in JWT authentication?

THEORY

An **access token** is **short-lived** (minutes) and is sent on every API call to prove identity; because it expires fast, a leaked one has a small blast radius. A **refresh token** is **long-lived** and used only to mint new access tokens, so it is stored more carefully and never sent on normal requests. This split lets you keep sessions long without keeping a powerful credential in flight constantly.

STRONG ANSWER

I issue a short-lived access token, usually around 15 minutes, that the client attaches to each request, and a long-lived refresh token that only hits a dedicated /refresh endpoint. The access token is stateless and verified by signature, so most requests need no database lookup, which keeps latency low. When it expires the client silently calls refresh to get a new pair instead of forcing the user to log in again. On the mobile side I keep the refresh token in secure storage like Keychain or Keystore, and the access token in memory, so even if memory is dumped the durable credential stays protected.

CODE · TypeScript

```
import jwt from 'jsonwebtoken';

const ACCESS_TTL = '15m';
const REFRESH_TTL = '30d';

export function issueTokens(userId: string) {
  const accessToken = jwt.sign({ sub: userId }, process.env.ACCESS_SECRET!, {
    expiresIn: ACCESS_TTL,
  });
  const refreshToken = jwt.sign({ sub: userId, typ: 'refresh' }, process.env.REFRESH_SECRET!, {
    expiresIn: REFRESH_TTL,
  });
  return { accessToken, refreshToken };
}

// middleware verifying the access token on each request
export function verifyAccess(req: any, res: any, next: any) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Missing token' });
  try {
    req.user = jwt.verify(token, process.env.ACCESS_SECRET!);
    next();
  } catch {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}
```

ARCHITECTURE / FLOW

```
Login -> { access(15m), refresh(30d) }

Normal call: Authorization: Bearer <access> -> verify sig

Access expired (401):
  client -> POST /refresh (refresh token)
    <- new { access, refresh }
  retry original request with new access
```


Q84

BACKEND & DATA

How does refresh-token rotation with token families help detect token theft?

THEORY

With **rotation**, every use of a refresh token invalidates it and issues a brand-new one, so a refresh token is single-use. All tokens descending from one login share a **family ID**, letting the server reason about the whole lineage. If an **already-used (revoked) refresh token is replayed**, that signals theft, and the server revokes the entire family, logging out both attacker and victim.

STRONG ANSWER

I built refresh-token rotation so each refresh call burns the old token and returns a new one tied to the same family. I store a hash of the current valid token per family in PostgreSQL with a used flag. The theft-detection trick is that if someone presents a refresh token that was already rotated, it means two parties hold the same token, so the legitimate user and a thief are racing. When that happens I revoke the whole family immediately, which forces a fresh login and cuts off the attacker. This gives long sessions the convenience users want while still containing a stolen refresh token to a single misuse.

CODE · TypeScript

```
import crypto from 'crypto';

const hash = (t: string) => crypto.createHash('sha256').update(t).digest('hex');

export async function rotateRefresh(presented: string) {
  const payload = jwt.verify(presented, process.env.REFRESH_SECRET!) as any;
  const row = await db.refreshTokens.findByHash(hash(presented));

  // replay of an already-used token => theft
  if (!row || row.used) {
    if (row) await db.refreshTokens.revokeFamily(row.familyId);
    throw new Error('Refresh reuse detected - family revoked');
  }

  await db.refreshTokens.markUsed(row.id); // burn old token
  const next = issueTokens(payload.sub);
  await db.refreshTokens.insert({
    familyId: row.familyId, // same lineage
    tokenHash: hash(next.refreshToken),
    used: false,
  });
  return next;
}
```

ARCHITECTURE / FLOW

```
Login => family F1, token R0
  use R0 -> R0 marked used, issue R1
  use R1 -> R1 marked used, issue R2 ...

Thief replays R0 (already used):
  server sees used=true -> REVOKE family F1
  both attacker & victim must log in again
```

Q85

BACKEND & DATA

How should passwords be hashed and application secrets be stored securely?

THEORY

Passwords must never be stored reversibly; you use a **slow, salted, adaptive hash like bcrypt, scrypt, or Argon2** so brute-forcing is expensive and identical passwords get different hashes. The **per-user salt** is built into these algorithms, defeating rainbow tables. Application secrets (signing keys, DB passwords) belong in **environment variables or a secrets manager**, never in source control.

STRONG ANSWER

For passwords I use bcrypt with a work factor around 10 to 12, which salts automatically and lets me raise cost as hardware improves. I only ever store the hash, and on login I compare with bcrypt.compare in constant time so I never expose whether the email or password was wrong. For secrets like my JWT signing keys and database credentials I keep them in environment variables loaded from a secrets manager, with .env git-ignored and separate keys per environment. I also rotate signing secrets when needed and keep access and refresh secrets distinct so compromising one does not compromise the other.

CODE · TypeScript

```
import bcrypt from 'bcrypt';

const COST = 12; // adaptive work factor

export async function registerUser(email: string, plain: string) {
  const passwordHash = await bcrypt.hash(plain, COST); // salt is embedded
  return db.users.insert({ email, passwordHash });
}

export async function verifyLogin(email: string, plain: string) {
  const user = await db.users.findByEmail(email);
  // compare even when user missing to avoid timing/enumeration leaks
  const ok = await bcrypt.compare(plain, user?.passwordHash ?? '$2b$12$invalidplaceholderhashvaluehere');
  if (!user || !ok) throw new Error('Invalid credentials');
  return user;
}

// secrets come from env, never hardcoded
const ACCESS_SECRET = process.env.ACCESS_SECRET;
if (!ACCESS_SECRET) throw new Error('Missing ACCESS_SECRET');
```

Q86

BACKEND & DATA

When would you choose PostgreSQL over MongoDB, and vice versa?

THEORY

PostgreSQL is a **relational store with strong schemas, joins, and ACID transactions**, ideal when data is structured and relationships and consistency matter. MongoDB is a **document store** that shines when the shape varies or you want to read a whole nested object in one hit without joins. The choice is about **access patterns and consistency needs**, not which is newer or faster in the abstract.

STRONG ANSWER

I reach for PostgreSQL when data is relational and correctness matters, like users, orders, and payments where I need foreign keys, transactions, and aggregate queries. I have also used pgvector inside Postgres for similarity search, which let me keep relational data and embeddings in one system instead of bolting on another database. I pick MongoDB when documents are self-contained and the schema is fluid, such as flexible event payloads or content that I read and write as one nested object. In practice I have run both in the same product, using Postgres as the source of truth and Mongo for the flexible, high-churn data, and the deciding factor is always the read and write pattern plus how strict the consistency needs to be.

CODE · SQL

```
-- PostgreSQL: relational integrity + transaction
BEGIN;
INSERT INTO orders (id, user_id, total_cents)
VALUES ('o_1', 'u_9', 4200);

UPDATE accounts SET balance_cents = balance_cents - 4200
WHERE user_id = 'u_9' AND balance_cents >= 4200;

-- foreign key guarantees the user exists; rollback on any failure
COMMIT;

-- pgvector similarity search alongside relational data
SELECT id, title
FROM documents
ORDER BY embedding <-> '[0.12, 0.04, ...]':vector
LIMIT 5;
```

ARCHITECTURE / FLOW

PostgreSQL		MongoDB
+-----+		+-----+
users <--FK--+		{ _id, user, items:[
orders		{sku, qty}], meta{}
accounts		} (one nested doc)
+-----+		+-----+
structured + ACID + joins		flexible schema, self-contained
-> source of truth		-> high-churn / variable shape

Q87**BACKEND & DATA****How did you use indexing and query optimization to cut API latency by 35%?****THEORY**

An **index is a sorted lookup structure** that lets the database find rows without scanning the whole table, turning $O(n)$ scans into $O(\log n)$ seeks. You target indexes at columns used in **WHERE, JOIN, and ORDER BY**, and use EXPLAIN ANALYZE to confirm the planner actually uses them. Beware over-indexing, which slows writes, and the **N+1 query pattern**, which a single batched query usually fixes.

STRONG ANSWER

I started by turning on slow query logging and running EXPLAIN ANALYZE on the hottest endpoints, which showed sequential scans on large tables filtered by user_id and status. I added composite indexes matching those filter and sort patterns, and a partial index for the common active-records case, which cut several queries from hundreds of milliseconds to single digits. I also killed an N+1 pattern in a list endpoint by replacing per-row lookups with one batched query plus a join, and added Redis caching for read-heavy reference data. Together the indexing, the N+1 fix, and caching brought the overall API latency down by about 35 percent, and I kept watching EXPLAIN output to make sure the plans stayed healthy as data grew.

CODE · SQL

```
-- before: seq scan on a large table, filtered + sorted
EXPLAIN ANALYZE
SELECT id, total_cents, created_at
FROM orders
WHERE user_id = 'u_9' AND status = 'active'
ORDER BY created_at DESC
LIMIT 20;

-- composite index matching WHERE columns then ORDER BY column
CREATE INDEX idx_orders_user_status_created
ON orders (user_id, status, created_at DESC);

-- partial index: most reads only touch active rows
CREATE INDEX idx_orders_active
ON orders (user_id, created_at DESC)
WHERE status = 'active';

-- re-run EXPLAIN ANALYZE: expect Index Scan, not Seq Scan
```

ARCHITECTURE / FLOW

```
Without index: Seq Scan -> read every row -> O(n)
[r1][r2][r3].....[rN]    slow

With composite index (user_id,status,created_at):
B-tree seek -> jump to matching range -> read 20
O(log n) + small range scan          fast
Result: hot endpoints ms-level, ~35% latency drop
```

Q88

BACKEND & DATA

What are the main Redis use cases, and how do you build a multi-instance-safe rate limiter with it?

THEORY

Redis is an **in-memory store** used for caching, session storage, queues, and rate limiting because reads and writes are sub-millisecond. For rate limiting across many app instances, the counter must live in **one shared place (Redis)**, **not per-process memory**, or each instance enforces its own limit. To stay correct under concurrency you run the increment and the limit check **atomically**, typically via a Lua script or INCR with EXPIRE.

STRONG ANSWER

I use Redis for caching expensive reads, for sessions and refresh-token bookkeeping, and for rate limiting. The reason in-memory per-process counters fail is that when you scale to several Node instances behind a load balancer, each one only sees its own slice of traffic, so the real limit becomes the limit times the number of instances. I solved this by keeping the counter in Redis keyed by client and window, and doing the increment plus expiry atomically with a small Lua script so two requests cannot interleave and overcount. That gives one consistent limit no matter how many instances are running, and because it is a single round trip to Redis it adds almost no latency.

CODE · TypeScript

```
import Redis from 'ioredis';
const redis = new Redis(process.env.REDIS_URL!);

// atomic fixed-window limiter: INCR then set TTL on first hit
const LUA = `
  local current = redis.call('INCR', KEYS[1])
  if current == 1 then redis.call('PEXPIRE', KEYS[1], ARGV[1]) end
  return current`;
```

CODE · TypeScript (continued)

```
export function rateLimit(limit: number, windowMs: number) {
  return async (req: any, res: any, next: any) => {
    const key = `rl:${req.ip}:${Math.floor(Date.now() / windowMs)}`;
    const count = (await redis.eval(LUA, 1, key, windowMs)) as number;
    if (count > limit) {
      res.setHeader('Retry-After', Math.ceil(windowMs / 1000));
      return res.status(429).json({ error: 'Too many requests' });
    }
    next();
  };
}
```

ARCHITECTURE / FLOW

Per-instance counters (WRONG): limit leaks x N

```
Inst A --+
Inst B --+--> [ Redis: INCR rl:ip>window ] <- one truth
Inst C --+      atomic INCR + PEXPIRE
              count > limit -> 429
Same limit enforced no matter how many instances run.
```

Q89

BACKEND & DATA

How do you handle concurrency, race conditions, and make write operations idempotent?

THEORY

A **race condition** happens when concurrent requests read-modify-write shared state and one overwrites the other, causing lost updates or double charges. You prevent it with **atomic operations**, **conditional updates**, or **locks** so only one writer wins. **Idempotency** means the same request applied twice has the same effect as once, which protects against client retries and network duplicates.

STRONG ANSWER

My main tools are doing the update atomically in the database and adding idempotency keys for unsafe operations. For something like decrementing inventory or a balance, I push the check into the UPDATE with a WHERE guard so the database enforces it in one atomic step rather than reading then writing in two round trips. For operations that must not run twice, like charging a card, I require the client to send an idempotency key, store the first result against that key, and return the cached result on any retry. I also use optimistic locking with a version column when needed, and Redis locks for cross-instance critical sections. This combination is what let my APIs stay correct under the retries that are normal on flaky mobile networks.

CODE · TypeScript

```
// idempotent create using a client-supplied key
export async function createPayment(req: any, res: any) {
  const key = req.headers['idempotency-key'] as string;
  if (!key) return res.status(400).json({ error: 'Idempotency-Key required' });

  const existing = await db.payments.findByIdemKey(key);
  if (existing) return res.status(200).json({ data: existing }); // safe retry

  // atomic balance check + decrement: DB enforces, no read-then-write race
  const debited = await db.query(
    `UPDATE accounts SET balance_cents = balance_cents - $1
     WHERE user_id = $2 AND balance_cents >= $1 RETURNING id`,
    [req.body.amount, req.body.userId]
  );
  if (debited.rowCount === 0) return res.status(409).json({ error: 'Insufficient funds' });
}
```

CODE · TypeScript (continued)

```
const payment = await db.payments.insert({ idemKey: key, ...req.body });
return res.status(201).json({ data: payment });
}
```

ARCHITECTURE / FLOW

```
Lost update (race):
R1 read bal=100 --\
R2 read bal=100 --/ both write 100-50 -> 50 (one lost!)

Atomic guarded UPDATE:
UPDATE ... SET bal=bal-50 WHERE bal>=50 -> DB serializes

Idempotency:
  retry same key -> return stored result, no double charge
```

Q90

BACKEND & DATA

What are the basics of error handling, input validation, and API security you always apply?

THEORY

Every input is untrusted, so you **validate and sanitize at the boundary** with a schema before it reaches business logic, rejecting bad requests with a clear 400. Errors should flow into a **single centralized handler** that returns a consistent shape and never leaks stack traces or internal details. Core hardening includes **CORS allow-lists, rate limiting, security headers, and not exposing whether a credential check failed on email vs password**.

STRONG ANSWER

I validate every request body, params, and query against a schema such as Zod at the edge, so malformed or malicious input is rejected with a 400 before it touches the database. All errors funnel through one Express error-handling middleware that maps known errors to proper status codes and returns a clean message, while logging the full detail server-side instead of sending stack traces to clients. For security I configure CORS with an explicit origin allow-list rather than a wildcard, add helmet for security headers, and apply the Redis rate limiter I built to blunt brute-force and abuse. I also keep auth responses generic, never reveal secrets in errors, and validate things like pagination limits so a client cannot ask for a million rows.

CODE · TypeScript

```
import { z } from 'zod';
import { Request, Response, NextFunction } from 'express';

const CreateUser = z.object({
  email: z.string().email(),
  age: z.number().int().min(13).max(120),
});

export function validate(schema: z.ZodTypeAny) {
  return (req: Request, res: Response, next: NextFunction) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      return res.status(400).json({ error: 'Invalid input', issues: result.error.issues });
    }
    req.body = result.data; // sanitized, typed
    next();
  };
}

// centralized error handler: consistent shape, no leaks
export function errorHandler(err: any, _req: Request, res: Response, _next: NextFunction) {
```

CODE · TypeScript (continued)

```
const status = err.status ?? 500;
console.error(err); // full detail stays server-side
res.status(status).json({ error: status === 500 ? 'Internal error' : err.message });
}
```

ARCHITECTURE / FLOW

```
Request
-> [CORS allow-list] -> [rate limit] -> [validate schema]
    bad origin 403      too many 429      bad body 400
-> [controller] --(throws)--> [central error handler]
    returns consistent JSON, logs detail, no stack leak
```

10

Payments, Releases, Testing & DevOps

Payments & webhooks, signing, staged rollout, Fastlane, CI/CD, tests, Sentry.

Q91–Q100

Q91

PAYMENTS, RELEASES, TESTING & DEVOPS

How do you integrate a payment gateway like Stripe or Razorpay into a React Native app safely?

THEORY

A payment flow should **never trust the client** to decide the amount or confirm success. The mobile app collects payment details through the gateway's SDK, but the **price, currency, and order are always created on your server**, which alone holds the secret API key. The client only receives a short-lived token or client secret to complete the charge.

STRONG ANSWER

The way I built it on Razorpay and Stripe was: the app asks our backend to create an order or PaymentIntent, the server computes the amount from the cart and returns a client secret, then the SDK opens the native payment sheet. The client never sends the price up; if it did, anyone could patch the app and pay one rupee for a thousand-rupee plan. After the user pays, the app shows a pending state, but I never mark the order paid from the client callback. The real confirmation comes from a server-side webhook, so the success screen is optimistic and the database is updated only when the gateway tells our server directly.

CODE · TypeScript

```
// client: ask server for a PaymentIntent, then confirm with the SDK
async function startCheckout(planId: string) {
  // server decides the real amount; client sends only the plan id
  const res = await api.post('/payments/intent', { planId });
  const { clientSecret } = res.data;

  const { error } = await confirmPayment(clientSecret, {
    paymentMethodType: 'Card',
  });
  if (error) {
    showError(error.message);
    return;
  }
  // optimistic UI only; truth comes from the webhook on the server
  navigate('PaymentProcessing', { planId });
}
```

ARCHITECTURE / FLOW

App	Server	Gateway
create intent		
----->	create w/ secret	
clientSecret	intent + secret	
<-----	<-----	
confirm payment (SDK)	----->	
	webhook (truth)	
	<-----	
	mark order paid	

Q92

PAYMENTS, RELEASES, TESTING & DEVOPS

What are webhooks and why are they the source of truth for payment status instead of the client response?

THEORY

A webhook is a server-to-server HTTP callback the gateway sends **directly to your backend** when an event happens, such as a successful charge or a failed renewal. It is the source of truth because it comes straight from the gateway over a **signed, authenticated channel**, bypassing the user's device entirely. Client callbacks can be lost, faked, or interrupted, so they are only hints for the UI.

STRONG ANSWER

I treat the webhook as the only event that is allowed to change money state in our database. The reason is that the client response can fail for completely legitimate reasons: the user's app crashes right after paying, they lose network, or they kill the app before the callback fires. If I relied on the client, those users would have paid but never been granted access. The webhook arrives independently and is signed with a secret, so I verify that signature, then update the order. I always verify the signature before doing anything, because an unauthenticated endpoint that grants subscriptions is an open door.

CODE · TypeScript

```
import express from 'express';
import Stripe from 'stripe';

const stripe = new Stripe(process.env.STRIPE_KEY!);

// raw body is required so the signature can be verified
export const webhook = express.raw({ type: 'application/json' });

export async function handler(req: express.Request, res: express.Response) {
  const sig = req.headers['stripe-signature'] as string;
  let event: Stripe.Event;
  try {
    event = stripe.webhooks.constructEvent(
      req.body, sig, process.env.STRIPE_WEBHOOK_SECRET!
    );
  } catch (err) {
    return res.status(400).send('invalid signature'); // reject forgeries
  }
  await processEvent(event); // only now do we trust it
  res.json({ received: true });
}
```

ARCHITECTURE / FLOW

```
Gateway ===signed POST===> Your Server
                        (out of band, never via the phone)

client callback ----> UI hint only (may be lost)
webhook             ----> DB write (source of truth)
```

Q93

PAYMENTS, RELEASES, TESTING & DEVOPS

How do you make webhook processing idempotent so you never double-charge or process the same event twice?

THEORY

Gateways guarantee **at-least-once delivery**, meaning the same webhook can arrive multiple times due to retries or timeouts. Idempotency means processing the same event repeatedly has the **same effect as processing it once**. The standard approach is to store each event's unique id and reject or no-op any id you have already handled, ideally inside a database transaction.

STRONG ANSWER

Every gateway event carries a stable unique id, so I have a `processed_events` table with that id as a primary key. When a webhook comes in, I try to insert the id first; if it violates the unique constraint, I know it is a duplicate and I return 200 immediately without touching anything. The insert and the business logic happen in one transaction, so a crash mid-way never leaves a half-applied state. This is exactly what saved us on Razorpay retries during a network blip where the same captured event arrived three times. Without it we would have extended a subscription three months instead of one.

CODE · TypeScript

```
async function processEvent(event: { id: string; type: string; data: any }) {
  await db.transaction(async (tx) => {
    // unique constraint on event_id makes this the dedup gate
    const inserted = await tx
      .insertInto('processed_events')
      .values({ event_id: event.id, type: event.type })
      .onConflict((c) => c.column('event_id').doNothing())
      .executeTakeFirst();

    if (Number(inserted.numInsertedOrUpdatedRows) === 0) {
      return; // already processed -> no-op, safe to ack
    }
    if (event.type === 'invoice.paid') {
      await extendSubscription(tx, event.data.subscriptionId, event.data.period);
    }
  });
}
```

ARCHITECTURE / FLOW

```
event id E1 arrives (1st time)
  INSERT E1 -> ok -> apply business logic -> ack 200

event id E1 arrives again (retry)
  INSERT E1 -> conflict -> skip logic -> ack 200
  (same end state, no double-charge)
```

Q94

PAYMENTS, RELEASES, TESTING & DEVOPS

How do you handle the subscription lifecycle states like trial, active, past_due, and canceled?

THEORY

A subscription is a **state machine** the gateway drives through webhooks, and your job is to mirror that state and gate feature access on it. The common path is trialing then active, but a failed renewal moves it to **past_due** where the gateway retries the card, and after retries are exhausted it becomes canceled. Access should follow the entitlement, not the raw payment, so trial and past_due users may still keep access for a grace window.

STRONG ANSWER

I model each state explicitly and let webhooks transition it rather than guessing on the client. During trial and active the user has full access; when a renewal fails the gateway moves them to past_due and I keep access alive during the dunning window while it retries the card, and I trigger an in-app banner asking them to update their payment method. If the retries fail it flips to canceled and I downgrade access at period end, not immediately, so they get what they paid for. The key lesson from billing work is to separate the subscription status from access logic, because past_due is not the same as locked out, and treating them the same churns users who would have happily updated a card.

CODE · TypeScript

```
type SubStatus = 'trialing' | 'active' | 'past_due' | 'canceled';

function hasAccess(s: { status: SubStatus; currentPeriodEnd: number }): boolean {
  const now = Date.now();
  switch (s.status) {
    case 'trialing':
    case 'active':
      return true;
    case 'past_due':
      return now < s.currentPeriodEnd; // grace window during dunning
    case 'canceled':
      return now < s.currentPeriodEnd; // keep until paid period ends
  }
}

// webhook handlers drive the transitions
const transitions: Record<string, SubStatus> = {
  'customer.subscription.trial_will_end': 'trialing',
  'invoice.paid': 'active',
  'invoice.payment_failed': 'past_due',
  'customer.subscription.deleted': 'canceled',
};
```

ARCHITECTURE / FLOW

```

trialing --invoice.paid--> active
  |                               |
  |           payment_failed      |
  v                               v
active <--invoice.paid-- past_due --retries fail--> canceled
                        (grace)           (access til period end)
```

Q95

PAYMENTS, RELEASES, TESTING & DEVOPS

What goes into app signing and store compliance for shipping a React Native app to the App Store and Play Store?

THEORY

Both stores require every build to be **cryptographically signed** so they can prove it came from you and was not tampered with. Android uses a **keystore** (and Google Play App Signing) while iOS uses **certificates and provisioning profiles** tied to your Apple Developer account. Beyond signing, stores run compliance checks on permissions, privacy declarations, and metadata before the build goes live.

STRONG ANSWER

On Android I keep an upload keystore secured and enrolled in Play App Signing, so Google holds the final signing key and I can recover if my upload key is ever lost. On iOS I manage distribution certificates and provisioning profiles, which I automate with Fastlane match so the whole team shares them from an encrypted repo instead of one laptop. The compliance side is what actually blocks releases: Apple wants an accurate privacy nutrition label and justification for sensitive permissions, and Google wants the Data Safety form and a target API level that meets their current minimum. I learned to treat the keystore as irreplaceable, because losing it without Play App Signing means you can never update that listing again.

CODE · Ruby (Fastlane)

```
# Fastlane match: shared, encrypted iOS signing identities
lane :certificates do
  match(type: "appstore", readonly: true)
end

lane :compliance_check do
  # fails the build early if privacy metadata is missing
  ensure_no_debug_code(text: "console.log")
  upload_to_app_store(
    submit_for_review: false,
    precheck_include_in_app_purchases: true,
    force: true # use metadata files as the source of truth
  )
end
```

ARCHITECTURE / FLOW

```
iOS: cert + provisioning profile -> signed .ipa -> App Store
      (managed via fastlane match, encrypted git repo)

Android: upload key -> Play App Signing -> final signing key
          (Google holds final key; upload key recoverable)

Store checks: permissions | privacy labels | target API | metadata
```

Q96

PAYMENTS, RELEASES, TESTING & DEVOPS

How do you manage a staged rollout and releases using Fastlane?

THEORY

A staged rollout releases a new version to a **small percentage of users first**, then widens the audience as confidence grows, so a regression hits 1 percent instead of everyone. Fastlane scripts the repetitive release steps such as building, signing, uploading, and setting the rollout fraction. The point is to **catch crashes in the wild before full exposure** and to be able to halt instantly.

STRONG ANSWER

I drive releases through Fastlane lanes so the process is identical every time and not dependent on someone remembering steps. I push to a small fraction first, usually 1 to 5 percent, then watch Sentry crash-free rates and key business metrics for a few hours before bumping to 10, 50, and finally 100 percent. If the crash-free rate dips below our threshold I halt the rollout from the same tooling rather than pushing a frantic hotfix. Owning this pipeline, the biggest win was making rollout fraction a parameter so promoting a release is one command, and the safety net is that a bad build never reaches the whole base at once.

CODE · Ruby (Fastlane)

```
# Fastlane: staged Play Store rollout with a parameterized fraction
lane :rollout do |options|
  fraction = options[:fraction] || 0.05 # start at 5%
  gradle(task: "bundle", build_type: "Release")
  upload_to_play_store(
    track: "production",
    rollout: fraction.to_s,
    aab: lane_context[SharedValues::GRADLE_AAB_OUTPUT_PATH],
    skip_upload_metadata: true
  )
  slack(message: "Rolled out v#{flutter_version} to #{fraction * 100}%")
end

# halt instantly if crash-free rate drops
lane :halt do
  upload_to_play_store(track: "production", rollout: "0")
end
```

ARCHITECTURE / FLOW

```
build+sign --> 1% --watch Sentry--> 10% --watch--> 50% --> 100%
      |               |               |
      crash spike?    crash spike?
      |               |               |
      v               v               v
      HALT (rollout = 0, no full exposure)
```

Q97**PAYMENTS, RELEASES, TESTING & DEVOPS****How would you set up CI/CD with GitHub Actions for a React Native mobile app?****THEORY**

CI runs **lint, type-check, and tests on every pull request** so broken code never merges, while CD builds and ships signed artifacts on tagged releases. For mobile you need **macOS runners for iOS builds** and to inject signing secrets securely rather than committing them. Caching dependencies keeps the pipeline fast enough that people actually wait for it.

STRONG ANSWER

My pipeline has a fast PR job that installs dependencies, runs ESLint, TypeScript, and the Jest suite, and it gates merging. On a release tag I trigger build jobs, with the iOS one on a macOS runner since Xcode is required, and signing material comes from encrypted GitHub secrets decrypted at build time, never from the repo. I lean on dependency and Gradle caching to keep PR feedback under a few minutes, because a slow pipeline gets ignored. The release job hands the signed build to Fastlane, which uploads to the stores, so the same lanes I run locally are what CI runs, which removes the works-on-my-machine class of release bugs.

CODE · YAML

```

name: ci
on:
  pull_request:
  push:
    tags: ['v*']
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: 'npm' }
      - run: npm ci
      - run: npm run lint
      - run: npx tsc --noEmit
      - run: npm test -- --ci --coverage
  ios-release:
    if: startsWith(github.ref, 'refs/tags/')
    needs: test
    runs-on: macos-14
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: bundle exec fastlane ios release
    env:
      MATCH_PASSWORD: ${ secrets.MATCH_PASSWORD }}

```

ARCHITECTURE / FLOW

```

PR opened
-> install -> lint -> tsc -> jest    (gate merge)

tag v*
-> test passes
-> ubuntu: build android aab
-> macos:  build ios ipa (Xcode)
-> fastlane -> stores (secrets injected, never in repo)

```

Q98

PAYMENTS, RELEASES, TESTING & DEVOPS

How do you write effective unit tests with Jest and React Native Testing Library?

THEORY

React Native Testing Library encourages testing **behavior from the user's perspective** rather than internal component state. You query elements the way a user finds them, by visible text or accessibility role, and assert on what renders, which makes tests resilient to refactors. Jest provides the test runner, assertions, and mocking for things like network calls and native modules.

STRONG ANSWER

I write tests around what the user sees and does, so I query by text or accessibility role and fire real events instead of poking at component internals. That means when I refactor a component's implementation the tests still pass as long as the behavior is the same, which is the whole point. I mock the network layer and any native modules so tests are deterministic and fast, and I use `findBy` queries for anything async like a loaded payment plan. For the billing screens I focused tests on the branches that matter, like rendering the `past_due` banner versus the active state, because those are the conditions where a bug actually costs money.

CODE · TypeScript

```
import { render, screen, fireEvent } from '@testing-library/react-native';
import { SubscriptionBanner } from './SubscriptionBanner';

describe('SubscriptionBanner', () => {
  it('prompts to update payment when past_due', () => {
    const onUpdate = jest.fn();
    render(<SubscriptionBanner status="past_due" onUpdate={onUpdate} />);

    // query the way a user would: by visible text / role
    expect(screen.getByText(/update your payment method/i)).toBeOnTheScreen();
    fireEvent.press(screen.getByRole('button', { name: /update/i }));
    expect(onUpdate).toHaveBeenCalledTimes(1);
  });

  it('shows nothing when active', () => {
    render(<SubscriptionBanner status="active" onUpdate={jest.fn()} />);
    expect(screen.queryByText(/update your payment/i)).toBeNull();
  });
});
```

Q99

PAYMENTS, RELEASES, TESTING & DEVOPS

How do you use Detox for end-to-end testing in a React Native app and how is it different from unit tests?

THEORY

Detox is a **gray-box end-to-end framework** that drives the real app on a simulator or device, tapping and typing like a user across whole flows. Unlike unit tests it exercises navigation, native modules, and the real bridge, so it catches integration bugs a component test cannot. Its key feature is being **synchronization-aware**: it waits for the app to be idle instead of using brittle fixed sleeps.

STRONG ANSWER

I use Detox for the critical end-to-end journeys, like onboarding through to a successful checkout, because those cross many screens and native boundaries that unit tests never touch. It launches the actual build on a simulator and interacts through accessibility ids, and because it knows when the app is busy with network or animations it waits automatically, which kills most of the flakiness you get from arbitrary sleeps. For payments I point it at the gateway's test mode so I can run a real card flow without charging anything. I keep the Detox suite small and focused on money paths and run it in CI nightly, since these tests are slower than Jest and you do not want them gating every PR.

CODE · TypeScript

```
describe('checkout flow', () => {
  beforeAll(async () => {
    await device.launchApp({ newInstance: true });
  });

  it('completes a subscription purchase in test mode', async () => {
    await element(by.id('plan-pro')).tap();
    await element(by.id('checkout-button')).tap();

    // Detox waits for the native payment sheet automatically
    await element(by.id('card-number')).typeText('4242424242424242');
    await element(by.id('pay-now')).tap();

    // assert the success screen rendered after the round trip
    await expect(element(by.text('Subscription active'))).toBeVisible();
  });
});
```

ARCHITECTURE / FLOW

```
unit (Jest+RNTL): one component, mocked deps, milliseconds
e2e (Detox):      whole app on simulator, real nav+native

tap plan -> tap checkout -> type card -> pay
           \_____ real bridge + network _____/
                   -> assert 'Subscription active'
```

Q100

PAYMENTS, RELEASES, TESTING & DEVOPS

How do you set up monitoring and crash reporting with Sentry, and how do you read a crash report?

THEORY

Sentry captures **unhandled crashes and errors with full stack traces**, breadcrumbs, device context, and the app release, so you can diagnose issues you cannot reproduce locally. For React Native you upload **source maps and native symbols** so minified stack traces map back to your real code. A crash report's value is in its context: the breadcrumbs leading up to the error and which release and device it hit.

STRONG ANSWER

I initialize Sentry early in the app with the release and environment tagged, and I make sure source maps and dSYMs are uploaded in the CI build, otherwise the stack traces are useless minified garbage. When a crash comes in, I read it top down: the exception type and message, then the stack frame in our code, then the breadcrumbs to see the user's last actions, and the release and OS version to scope who is affected. During staged rollouts this is exactly what I watch, the crash-free session rate per release, and a spike is my signal to halt. The breadcrumbs are usually what crack it, because seeing the payment confirm event right before a null crash points straight at the offending screen.

CODE · TypeScript

```
import * as Sentry from '@sentry/react-native';

Sentry.init({
  dsn: process.env.SENTRY_DSN,
  release: `app@${appVersion}`, // ties crashes to a staged rollout
  environment: __DEV__ ? 'dev' : 'production',
  tracesSampleRate: 0.2,
  beforeSend(event) {
    // never ship card data to the crash reporter
    delete event.extra?.cardNumber;
    return event;
  },
});

// add context so a crash report is actually readable later
Sentry.addBreadcrumb({ category: 'payment', message: 'confirm tapped' });
Sentry.setTag('subscription_status', 'past_due');
```

ARCHITECTURE / FLOW

```
Crash report, read top-down:
+-----+
| TypeError: undefined is not an object | <- what
| at CheckoutScreen.tsx:42 (your frame) | <- where
| breadcrumbs: view plan > confirm tapped | <- how
| release app@2.3.0 iOS 17.4 device X | <- who/scope
+-----+
spike in crash-free rate -> halt rollout
```